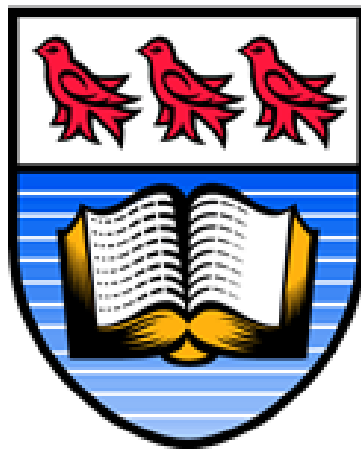


Department of Electrical and Computer
Engineering
University of Victoria



MyFOC
A From-Scratch Field-Oriented Control
Library with Application in Rehabilitation

Graduate Project – Master of Electrical & Computer Engineering

Submitted: [Month Day], 2026

Hoang Tuan Kiet Vu –

Contents

List of Figures	iv
List of Tables:	v
Symbols.....	vi
Acronyms.....	vii
Abstract.....	viii
1. Introduction	1
1.1. Background and Motivation	1
1.2. Objectives and Contributions.....	1
1.3. Scope and Limitations.....	2
2. Theoretical Background	3
2.1. The Three-Phase PMSM.....	3
2.2. Reference-Frame Transformations	4
2.2.1. Clarke Transformation ($abc \rightarrow \alpha\beta$)	4
2.2.2. Park Transformation ($\alpha\beta \rightarrow dq$)	5
2.2.3. PMSM Model in the dq Frame	5
2.3. Space-Vector Pulse-Width Modulation	6
2.3.1. Sinusoidal PWM	6
2.3.2. Space-Vector PWM with Min-Max Injection.....	6
2.4. Quadrature Encoder Decoding.....	7
2.5. Cascaded Control of a PMSM	7
2.5.1. Architecture.....	7
2.5.2. Cross-Axis Decoupling Feed-Forward	7
2.5.3. Internal-Model-Control (IMC) Tuning of the Current Loops.....	8
2.5.4. Anti-Windup and Saturation	8
2.6. Impedance Control.....	8
2.7. SimpleFOC as a Reference Implementation.....	9
3. System Design and Architecture	10
3.1. Hardware Overview	10
3.2. Software Architecture	12
3.3. Control-System Architecture	14
3.4. Safety Architecture	14
4. Implementation.....	15
4.1. Mathematical Kernel (MyFOCMath)	15

4.1.1.	Sin/Cos Lookup Table	15
4.1.2.	Clarke, Park, Inverse Park, Inverse Clarke	16
4.1.3.	Space-Vector PWM	16
4.2.	PWM Driver (MyBLDCDriver3PWM).....	16
4.3.	Quadrature Encoder Decoder (MyEncoder)	17
4.4.	Current Sensing (MyCurrentSense3PWM)	17
4.4.1.	Zero calibration.....	18
4.4.2.	Reconstruction of i_c	18
4.4.3.	ADC clock acceleration	18
4.5.	PID and Low-Pass Filter (MyPID, MyLPF).....	18
4.6.	Motor Object and FOC Pipeline (MyBLDCMotor)	19
4.7.	Sensor Alignment (alignSensor)	19
4.8.	Motor Parameter Identification.....	20
4.8.1.	Resistance	20
4.8.2.	Inductance	20
4.8.3.	Flux Linkage	21
4.9.	Impedance Controller.....	21
4.10.	Data Logging and Monitoring	21
5.	Experimental Setup.....	22
5.1.	Bench Configuration.....	22
5.2.	Test Procedures.....	26
5.2.1.	Parameter Identification Procedure.....	26
5.2.2.	FOC Validation Procedure.....	26
5.2.3.	Clinical Exercise-Mode Procedure	27
5.3.	Data Logging and Plotting Pipeline.....	27
6.	Results and Validation.....	27
6.1.	Compile-Time Verification.....	27
6.2.	Hardware Bring-Up and Auto-Detected Parameters	27
6.3.	Identified Motor Parameters	28
6.4.	Functional Verification of the FOC Pipeline	30
6.5.	Application: Robotic Rehabilitation Devices	30
6.5.1.	Clinical Exercise Modes: Five-Mode Validation	31
7.	Limitations and Future Work	37
7.1.	Limitations	37
7.2.	Future Work.....	39

7.3. Path to Clinical Deployment.....	40
8. Conclusion.....	42
References.....	44
Appendix A. Source-File Organisation.....	45
Appendix B. Mathematical Derivations	46
B.1 Clarke Transformation (Amplitude-Invariant Form)	46
B.2 Park Transformation.....	46
B.3 PMSM Torque Equation.....	47
B.4 IMC Tuning of the First-Order Current-Loop Plant.....	47
B.5 Impedance Dynamics for a Single-Degree-of-Freedom Load	48

List of Figures

Figure 1: Complete field-oriented control (FOC) signal flow as implemented in MyFOC. The forward path runs the d- and q-axis current PI controllers, inverse Park, inverse Clarke and min-max space-vector PWM into the DRV8302 inverter; the rotor electrical angle θ_e , derived from the AMT103 encoder, rotates both the forward and the feedback transforms, and the measured phase currents are returned through Clarke and Park to close the current loop. The cross-axis decoupling feed-forward is added to the PI outputs. In the voltage-mode fallback used for the experiments in this project (dashed), v_q is taken directly from the impedance or torque law and the current loop is bypassed.	4
Figure 2: Electrical diagram of the setup.	22
Figure 3: IPOWER GM4108-120T Mount 3D design.	23
Figure 4: Pendulum Arm 3D design.	24
Figure 5: The final setup with everything connected.....	24
Figure 6: Close up of the connected gimbal with encoder.....	25
Figure 7: The power benches used for this project.	25
Figure 8: Setup for the isometric test (Test 1). The controller holds the lever level with a stiff virtual spring while calibrated weights are hung; the holding voltage and the known weight together give a torque measurement that is independent of the commanded control law.	31
Figure 9: Isometric test (Test 1). (a) Holding q-axis voltage steps up as weight is added. (b) Deadweight calibration: the true gravitational torque (50–150 g) follows a 0.032 N·m/V line, about 1.6× below the voltage-mode reconstruction $K_t/R = 0.0511$ N·m/V; the 200 g point is saturated (about 40° droop) and excluded.	32
Figure 10: Setup for the isotonic test (Test 2). With the position spring disabled a constant torque is commanded and the lever rotates until the gravitational torque of the fixed 200 g load balances it; the settle angle is recorded against the commanded torque.	33
Figure 11: Isotonic test (Test 2). (a) Reconstructed torque commanded in flat steps. (b) The constant torque lifts a fixed 200 g load to a progressively larger settle angle (about 28° to 62°) as the command is increased from 0.10 to 0.30 N·m.....	33
Figure 12: Setup for the isokinetic test (Test 3). A velocity set-point of ± 2 rad/s drives the arm at constant speed through continuous rotation.....	34

Figure 13: Isokinetic test (Test 3). Measured shaft velocity (raw and 0.4 s moving average) against the ± 2 rad/s set-point; the mean tracks the command ($+2.20 / -2.02$ rad/s) while the instantaneous estimate is quantisation-noisy.	34
Figure 14: Setup for the adaptive test (Test 4). The operator pulls the arm slowly and then quickly; the effective stiffness and ceiling scale with the measured effort (low-pass-filtered speed while pushing outward).	35
Figure 15: Adaptive test (Test 4) at the deployed baseline ($K_{\text{base}} = 0.5$ N·m/rad, $L_{\text{base}} = 0.05$ N·m, $A = 2.0$). (a) Effort and K_{eff} against time: with adaptation off ($A = 0$) effort spikes but K_{eff} stays at baseline; with $A = 2.0$ it scales with effort. (b) K_{eff} versus effort — the adaptive points match the law $K_{\text{base}} \cdot (1 + A \cdot \text{effort} / v_{\text{ref}})$ to an RMS of 3.3×10^{-4} N·m/rad ($n = 1362$).	36
Figure 16: Setup for the back-drive test (Test 5). The motor is de-energised ($v_q = 0$); the free arm is released to ring down and small weights are added to find the breakaway torque.	36
Figure 17: Back-drive / transparency test (Test 5). (a) The de-energised arm swings freely with light damping ($\zeta \approx 0.02\text{--}0.05$). (b) Static breakaway torque for the 15 / 20 / 32 g weights, 0.018–0.038 N·m.	37

List of Tables:

Table 1: Hardware configuration.	11
Table 2: Pin assignments.	11
Table 3: Module decomposition of the MyFOC library.	12
Table 4: Quadrature decoding logic.	17
Table 5: Diagnostic outputs from the as-built bench.	28
Table 6: Empirical motor parameters from the identification procedure.	29
Table 7: Indicative per-axis parts cost (USD) for the bench presented here and two clinical-grade configurations. Excludes certification, software licences, and labour.	41

Symbols

Symbol	Meaning
θ_m, ω_m	Mechanical angle (rad), mechanical angular velocity (rad/s)
θ_e, ω_e	Electrical angle (rad), electrical angular velocity (rad/s); $\omega_e = pp \cdot \omega_m$
pp	Number of pole pairs
v_a, v_b, v_c	Three-phase stator voltages (V)
i_a, i_b, i_c	Three-phase stator currents (A)
v_α, v_β	Stator voltages in the stationary (Clarke) reference frame
i_α, i_β	Stator currents in the stationary reference frame
v_d, v_q	Direct- and quadrature-axis voltages (rotor frame, V)
i_d, i_q	Direct- and quadrature-axis currents (rotor frame, A)
R, R_{phase}	Stator phase resistance (Ω)
L_d, L_q	Direct- and quadrature-axis inductance (H)
λ_{PM}	Permanent-magnet flux linkage (Wb = V·s/rad)
$K_t = \left(\frac{3}{2}\right) \cdot pp \cdot \lambda_{PM}$	Torque constant (N·m/A)
τ	Electromagnetic torque (N·m)
V_{DC}, V_{bus}	DC bus voltage of the inverter (V)
K, B, θ_{eq}	Virtual stiffness, damping and equilibrium of the impedance controller

Acronyms

Acronym	Expansion
BLDC	Brushless Direct Current (motor)
PMSM	Permanent-Magnet Synchronous Machine
FOC	Field-Oriented Control
PWM	Pulse-Width Modulation
SVPWM	Space-Vector Pulse-Width Modulation
SPWM	Sinusoidal Pulse-Width Modulation
MMF	Magneto-Motive Force
EMF	Electro-Motive Force (back-EMF)
PI / PID	Proportional-Integral / Proportional-Integral-Derivative controller
IMC	Internal Model Control (PI tuning method)
LPF	Low-Pass Filter
ADC	Analogue-to-Digital Converter
CPR	Counts Per Revolution (encoder, post-quadrature)
PPR	Pulses Per Revolution (encoder, pre-quadrature)
MCU	Microcontroller Unit
AVR	Advanced Virtual RISC (Atmel/Microchip 8-bit MCU family)
ISR	Interrupt Service Routine
DRV	Driver IC (Texas Instruments DRV8302 used here)

Abstract

Field-Oriented Control (FOC) is the dominant high-performance control strategy for permanent-magnet synchronous machines (PMSM) and brushless DC (BLDC) motors. By transforming three-phase stator quantities into a rotor-synchronous direct/quadrature (dq) reference frame, FOC reduces the control problem to two decoupled DC loops, enabling smooth, low-ripple torque production across the entire speed range.

This project presents MyFOC, a Field-Oriented Control library implemented entirely from first principles on the Arduino UNO (ATmega328P, 16 MHz, 32 KB flash, 2 KB SRAM) [1] and the Texas Instruments DRV8302 three-phase gate driver [2]. The library provides quadrature encoder decoding, 4-quadrant 31.4 kHz synchronized PWM generation, low-side current sensing, the forward and inverse Clarke [3] and Park [4] transformations, space-vector PWM with min–max common-mode injection [5], [6], sensor alignment with pole-pair verification, cascaded current and velocity loops with Internal-Model-Control (IMC) tuning [7], cross-axis decoupling feed-forward, automated identification of the four PMSM electrical parameters ($R, L_d, L_q, \lambda_{PM}$), and a closed-loop impedance controller [8], [9] that serves as the substrate for a human-interaction application.

The library is deployed on a small 11-pole-pair gimbal motor instrumented with an AMT103 incremental encoder [10] (2048 pulses per revolution, giving 8192 counts after $4\times$ decoding). On identical hardware, MyFOC is verified stage by stage against the behaviour expected of a correct FOC implementation, taking the widely-used SimpleFOC library [11] as a documented reference point. Although the libraries differ in capabilities (built-in identification, IMC tuning, and impedance control in MyFOC; multi-platform portability in SimpleFOC), the bring-up tests confirm that MyFOC reproduces a fully functional FOC commutation firmware suitable as a platform for human-interaction applications. All results reported here were obtained with the firmware in its voltage-mode commutation path: the rotor angle closes the commutation loop and the impedance law closes a position/velocity loop, but phase current is not fed back and the q-axis voltage is commanded directly. The closed-loop current controller - the current PI loops, IMC tuning and cross-axis decoupling - is implemented and was brought up on the bench, but it was not exercised in these results and is identified as future work (Section 7). The complete MyFOC firmware compiles to approximately 21 KB of program memory and consumes no static SRAM beyond per-object instance state, fitting comfortably within the ATmega328P budget while leaving headroom for application code.

The library is exercised on a force-rendering and torque-shaping task, described in full in Section 6.5: an upper-limb rehabilitation trainer renders a programmable virtual spring–damper field defined by stiffness K , damping B and equilibrium θ_{eq} , with a hardware-enforced torque ceiling for clinical safety and a velocity-gated adaptive-scaling layer that increases resistance when the patient pushes hard and relaxes it when they push gently. Multiple clinically meaningful rehabilitation modes are also implemented by changing the parameters without re-flashing the software.

1. Introduction

1.1. Background and Motivation

Robotic systems intended for direct physical interaction with human beings - collaborative manipulators, exoskeletons, surgical robots, and rehabilitation devices - share a common requirement: they must produce smooth, accurately-controlled torque at low speeds, often with large dynamic range and sometimes with the actuator effectively back driven by the operator. Three-phase permanent-magnet synchronous machines, when driven by FOC, are well suited for these applications because the rotor-frame transformation eliminates the commutation-induced torque ripple that plagues trapezoidal-driven brushless DC motors and brings the inner-current dynamics into a form (a stable first-order DC plant) for which classical linear control is sufficient.

Field-Oriented Control has been studied since the late 1960s and matured into a textbook subject by the 1990s [12], [13], but its software realization on inexpensive hardware became commonplace only with the rise of capable 32-bit ARM Cortex-M microcontrollers in the 2010s. Open-source motor-control firmware accelerated this trend significantly: in particular, the SimpleFOC project [11] provides a polished, well-documented Arduino-compatible library that runs on hardware ranging from the AVR-based UNO to STM32 and ESP32 platforms, and has become a common reference in undergraduate motor-control courses and small-scale research.

Several application domains have emerged that share a need for smooth, programmable, low-friction torque control. Robotic rehabilitation devices for stroke, spinal cord injury and orthopedic recovery have become an active clinical and research area; the dominant low-level control paradigm here is impedance control, introduced by Hogan in 1985 [8], in which the actuator is commanded to render a programmable virtual mechanical impedance (typically a spring-damper field) rather than to track a position or velocity reference. Impedance control preserves passive compliance (the patient can always overpower the device), is robust to model uncertainty, and allows clinically-motivated rehabilitation modes - isometric strength training, isotonic resistance, isokinetic constant-velocity motion - to be expressed cleanly as different choices of stiffness K and damping B [9].

The work reported in this project sits at the intersection of these threads: a from-scratch FOC firmware that is pedagogically transparent and benchmarkable, a low-cost hardware platform (Arduino UNO + DRV8302) accessible to undergraduate laboratories, and a rehabilitation application that exercises the impedance-control layer of the stack.

1.2. Objectives and Contributions

The project pursues four explicit objectives:

- To design and implement complete FOC firmware (encoder decoder, PWM generator, low-side current-sense interface, dq mathematics, sensor-alignment routine, pole-pair check, PID/LPF, cascaded current and velocity loops, SVPWM modulator, and decoupling feed-forward) on the Arduino UNO + DRV8302 platform.

- To extend the firmware with an identification procedure that measures R, L_d, L_q and λ_{PM} with no auxiliary instrumentation beyond the motor itself, and an IMC-based automatic current-loop tuner that consumes those parameters.
- To realize an application on top of the impedance kernel: an upper-limb rehabilitation controller with a velocity-gated adaptive-scaling layer, tested via a serial command interface that allows runtime adjustment.
- To validate the firmware on a IPOWER GM4108-120T gimbal motor (11 pole pairs, AMT103 encoder, 12 V supply): to verify each algorithmic block on identical hardware against the behaviour expected of a correct FOC implementation, taking the widely-used SimpleFOC library [11] as a documented reference point (Section 6.4); and to validate the rehabilitation application across five clinically recognised exercise modes — isometric, isotonic, isokinetic, adaptive-resistance and transparent back-drive (Section 6.5).

Beyond the working artifact, the project contributes:

- A complete mathematical derivation of each transformation block, written in a form designed to be easily improved for future desires.
- A documented and quantified analysis of several implementation defects encountered during development - most notably a 90-degree alignment-offset error rooted in the geometric relationship between U_q application and rotor d-axis settling - that are not always made explicit in motor-control texts.
- A reusable test methodology for PMSM parameter identification on ADC-limited microcontrollers, including the boost of the AVR ADC clock from 125 kHz to 1 MHz required to resolve the L/R time constant of small gimbal motors.
- A quantitative validation methodology that expresses the rehabilitation application's claims as analytic predictions and confirms them with standard statistical tests - least-squares regression with R^2 for the stiffness and damping laws, and RMS-residual analysis of the adaptive-scaling law (Section 6.5).

1.3. Scope and Limitations

This work is explicitly scoped as follows:

- The target machine class is a small surface-PMSM ($L_d \approx L_q$) such as a hobby gimbal motor; the firmware does not attempt field-weakening, interior-PMSM handling, or salient-pole maximum-torque-per-ampere control.
- The DC bus voltage is fixed at 12 V; bus-voltage feed-forward and dynamic V_{bus} measurement is not implemented.
- Position feedback is from an incremental optical quadrature encoder; absolute encoders, magnetic angle sensors and sensorless observers are out of scope.
- The inner current-loop sample rate is bounded above by the ATmega328P's ADC and floating-point performance; the design targets approximately 1.5 kHz nominal loop execution rate at the current axis, which is conservative for the 31.4 kHz PWM carrier but appropriate for the host MCU. All experimental results in this project are obtained in voltage-mode commutation, with the current/torque loop open (no current feedback); the closed-loop current controller is implemented and was brought up on

the bench, but its experimental validation is out of scope and is treated as future work (Section 7).

- Human clinical evaluation of the rehabilitation application is out of scope; the focus is on the engineering of the actuation layer and its programmable interface.

2. Theoretical Background

2.1. The Three-Phase PMSM

A permanent-magnet synchronous machine consists of a three-phase stator winding distributed at 120 electrical degrees around the bore, and a rotor carrying permanent magnets that establish a fixed magnetic flux λ_{PM} linked into each stator phase as a sinusoidal function of the rotor's electrical angle θ_e . The relationship between mechanical and electrical angles is determined by the number of magnetic pole pairs (pp) on the rotor: $\theta_e = pp \cdot \theta_m$. The motor used in this work has pp 11, so one mechanical revolution corresponds to eleven electrical cycles.

Under the assumption of a balanced winding, sinusoidal back-EMF (a well-designed surface PMSM), magnetic linearity, and a star (Y) connection with no neutral wire, the per-phase voltage equations are:

$$v_a = R \cdot i_a + L \cdot \frac{di_a}{dt} + e_a$$

$$v_b = R \cdot i_b + L \cdot \frac{di_b}{dt} + e_b$$

$$v_c = R \cdot i_c + L \cdot \frac{di_c}{dt} + e_c$$

where R is the stator phase resistance, L is the per-phase self-inductance (mutual inductances are absorbed into an effective self-inductance for the balanced case), and e_a, e_b, e_c are the per-phase back-EMFs:

$$e_a = -\lambda_{PM} \cdot \omega_e \cdot \sin(\theta_e)$$

$$e_b = -\lambda_{PM} \cdot \omega_e \cdot \sin(\theta_e - \frac{2\pi}{3})$$

$$e_c = -\lambda_{PM} \cdot \omega_e \cdot \sin(\theta_e + \frac{2\pi}{3})$$

The electromagnetic torque developed by the machine is obtained from the power balance and, after writing the back-EMF in terms of the stator currents, reduces (in the rotor reference frame derived below) to the compact form:

$$\tau = \frac{3}{2} \cdot pp \cdot \lambda_{PM} \cdot i_q = K_t \cdot i_q$$

for a surface PMSM ($L_d \approx L_q$, so the reluctance term vanishes). The torque constant $K_t = \frac{3}{2} \cdot pp \cdot \lambda_{PM}$ links the rotor-frame quadrature current directly to the mechanical torque, which is the foundation of every current-mode FOC design including the one developed here.

2.2. Reference-Frame Transformations

Direct control of the three-phase sinusoidal currents at runtime is unattractive: the controller would have to track a moving target whose frequency varies with rotor speed. The classical remedy is a two-step transformation of the abc quantities first into a two-axis stationary frame $\alpha\beta$ (Clarke), then into a two-axis frame dq that rotates synchronously with the rotor (Park). In dq , the steady-state quantities are constant, so PI control becomes straightforward. Figure 1 shows the complete FOC signal flow assembled from these transformations together with the current and outer-loop controllers; the remainder of this chapter derives each block in turn, and the inner loop `loopFOC()` realises the same flow in firmware.

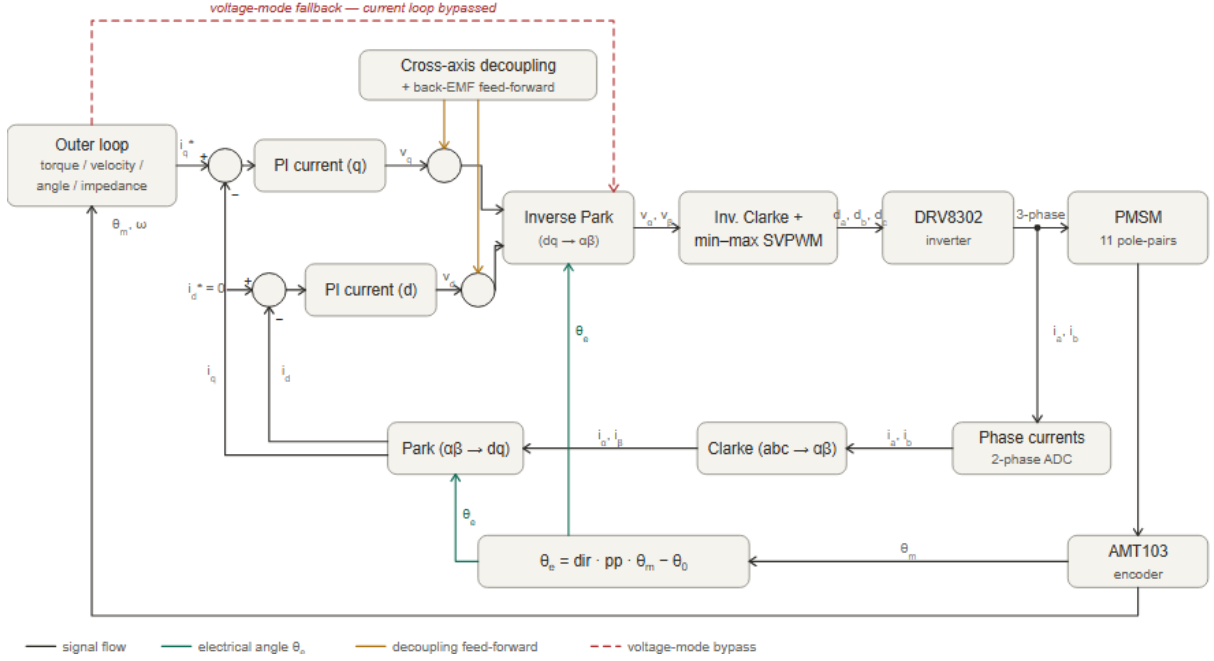


Figure 1: Complete field-oriented control (FOC) signal flow as implemented in MyFOC. The forward path runs the d - and q -axis current PI controllers, inverse Park, inverse Clarke and min-max space-vector PWM into the DRV8302 inverter; the rotor electrical angle θ_e , derived from the AMT103 encoder, rotates both the forward and the feedback transforms, and the measured phase currents are returned through Clarke and Park to close the current loop. The cross-axis decoupling feed-forward is added to the PI outputs. In the voltage-mode fallback used for the experiments in this project (dashed), v_q is taken directly from the impedance or torque law and the current loop is bypassed.

2.2.1. Clarke Transformation ($abc \rightarrow \alpha\beta$)

The three stator axes are physically displaced by 120 mechanical degrees. Choosing the α -axis to coincide with phase A and the β -axis to lie 90 electrical degrees ahead in the counter-clockwise sense, the unit vectors of the three phase axes resolved in the $\alpha\beta$ plane are

$(1, 0)$, $(-\frac{1}{2}, \frac{\sqrt{3}}{2})$ and $(-\frac{1}{2}, -\frac{\sqrt{3}}{2})$. The α and β components of any three-phase quantity are the projections onto these axes, scaled by a convention factor k [3]:

$$v_\alpha = k \cdot (v_a \cdot 1 + v_b \cdot \left(-\frac{1}{2}\right) + v_c \cdot \left(-\frac{1}{2}\right))$$

$$v_\beta = k \cdot (v_a \cdot 0 + v_b \cdot \left(\frac{\sqrt{3}}{2}\right) + v_c \cdot \left(-\frac{\sqrt{3}}{2}\right))$$

Two scaling conventions are in common use. The amplitude-invariant convention $k = \frac{2}{3}$ yields v_α and v_β whose magnitude equals the peak of the original abc sinusoids, and is the convention adopted by SimpleFOC [11], most automotive supplier libraries and most pedagogical textbooks [12], [13]. The power-invariant convention $k = \sqrt{\frac{2}{3}}$ preserves the total instantaneous three-phase power between the abc and $\alpha\beta$ frames; it is preferred in analytical treatments of machine impedance but is less common in practical firmware. MyFOC uses the amplitude-invariant convention throughout for comparability with SimpleFOC:

$$v_\alpha = \left(\frac{2}{3}\right) \cdot \left(v_a - \frac{v_b + v_c}{2}\right)$$

$$v_\beta = \frac{v_b - v_c}{\sqrt{3}}$$

For balanced abc systems ($v_a + v_b + v_c = 0$), these expressions reduce to $v_\alpha = v_a$ and $v_\beta = \frac{v_a + 2v_b}{\sqrt{3}}$. The unbalanced form is implemented to remain valid during transients, line-to-line faults and the open-loop test sequences used by the identification routines.

2.2.2. Park Transformation ($\alpha\beta \rightarrow dq$)

The Park transformation [4] rotates the $\alpha\beta$ frame by $-\theta_e$ (i.e. into a frame that co-rotates with the rotor's d-axis) so that DC quantities in the steady state correspond to fixed stator-current vectors locked to the rotor magnet:

$$[v_d] = [\cos\theta_e \quad \sin\theta_e] [v_\alpha]$$

$$[v_q] = [-\sin\theta_e \quad \cos\theta_e] [v_\beta]$$

By convention v_d (the direct-axis component) is aligned with the rotor's magnet (N-pole) and contributes only to magnetisation; v_q (the quadrature component) is 90 electrical degrees ahead and is the torque-producing component. The inverse transformation is the corresponding $+\theta_e$ rotation:

$$[v_\alpha] = [\cos\theta_e \quad -\sin\theta_e] [v_d]$$

$$[v_\beta] = [\sin\theta_e \quad \cos\theta_e] [v_q]$$

The geometric content of these formulas is that a quadrature-axis voltage applied at electrical angle θ_e produces a stator MMF vector at angle $\theta_e + \frac{\pi}{2}$ in the $\alpha\beta$ plane. This single geometric fact has direct consequences for the sensor-alignment procedure described in Section 4.7 and was the origin of one of the implementation defects documented in Section 4.7.

2.2.3. PMSM Model in the dq Frame

Applying Clarke and Park transformations to the three-phase voltage equations and gathering terms, the PMSM dynamic model in the rotor reference frame is [12], [13]:

$$v_d = R \cdot i_d + L_d \cdot \frac{di_d}{dt} - \omega_e \cdot L_q i_q$$

$$v_q = R \cdot i_q + L_q \cdot \frac{di_q}{dt} + \omega_e \cdot (L_d \cdot i_d + \lambda_{PM})$$

Two features of this model are pivotal for control design. First, the d-axis equation contains a cross-coupling term $-\omega_e \cdot L_q \cdot i_q$ that depends on the q-axis current, and the q-axis equation contains the dual term $+\omega_e \cdot L_d \cdot i_d$ together with the speed-proportional back-EMF $\omega_e \cdot \lambda_{PM}$. From the PI controller's perspective these are disturbances; they are linear in measured quantities (ω_e, i_d, i_q) and known parameters (L_d, L_q, λ_{PM}) and can therefore be cancelled by a feed-forward term - see Section 2.5.2.

Second, with the cross-coupling cancelled, each axis becomes a stable first-order plant of the form $v = R \cdot i + L \cdot \frac{di}{dt}$, whose transfer function in the Laplace domain is $\frac{i(s)}{v(s)} = \frac{1}{L \cdot s + R}$. This is the canonical first-order target that the Internal-Model-Control rule of Section 2.5.3 is designed to invert.

2.3. Space-Vector Pulse-Width Modulation

Once the inverse Park and inverse Clarke transformations have produced reference phase voltages v_a, v_b, v_c , the inverter must reproduce them via PWM. Two practical strategies exist:

2.3.1. Sinusoidal PWM

In sinusoidal PWM each phase reference is offset to $\frac{V_{DC}}{2}$ and divided by V_{DC} to yield a duty cycle in $[0, 1]$. This is mathematically transparent but limits the achievable peak phase voltage to $\frac{V_{DC}}{2}$ and therefore the maximum line-to-line voltage to $V_{DC} \cdot \frac{\sqrt{3}}{2} \approx 0.866 \cdot V_{DC}$; the inverter's full DC bus is never fully utilized.

2.3.2. Space-Vector PWM with Min-Max Injection

Because the three motor windings are connected as a floating star with no neutral wire, the common-mode component of (v_a, v_b, v_c) does not appear across any winding. Adding the same offset to all three phases is therefore invisible to the motor but allows the modulator to reduce the peak excursion of any one phase. The classical sector-based SVPWM formulation [5], derived by representing the desired voltage as a sum of two adjacent active vectors and a zero vector, is mathematically equivalent (within rounding) to the more compact min-max injection scheme [6]:

$$v_{offset} = \frac{\max(v_a, v_b, v_c) + \min(v_a, v_b, v_c)}{2}$$

$$v'_a = v_a - v_{offset} \text{ (and analogously for } b, c)$$

$$duty_x = \left(\frac{v'_x}{V_{DC}}\right) + 0.5$$

The result extends the peak achievable phase voltage to $\frac{V_{DC}}{\sqrt{3}} \approx 0.577 \cdot V_{DC}$ and the line-to-line voltage to V_{DC} , a 15.5 % improvement in modulation depth over sinusoidal PWM. MyFOC uses the min-max formulation for all closed-loop FOC operation; the sinusoidal form is provided as a separately-callable function for instructional and comparison purposes.

2.4. Quadrature Encoder Decoding

An incremental quadrature encoder produces two square-wave channels (A, B) that are 90 electrical degrees out of phase [10]. By detecting every edge on both channels, four counts are obtained per encoder line, giving an effective resolution of $4 \times \text{PPR}$ counts per revolution (CPR). The decoder must additionally determine the direction of motion, which is encoded in the relative phase: a transition $(A, B) = (0, 0) \rightarrow (1, 0) \rightarrow (1, 1) \rightarrow (0, 1) \rightarrow (0, 0)$ indicates one direction, and the reverse sequence the other.

On the ATmega328P [1], only digital pins 2 and 3 support hardware external interrupts (INT0, INT1) at the resolution needed to capture every encoder edge without missing transitions at maximum motor speed. MyFOC requires the encoder to be wired to these two pins and configures CHANGE-mode interrupts on each. The ISR for each channel reads both channel states and updates a 32-bit pulse counter according to a logic table derived in Section 4.3.

2.5. Cascaded Control of a PMSM

2.5.1. Architecture

A FOC drive is organized as a cascade of nested loops. From innermost to outermost:

- Inner current loops (PI on i_d and on i_q): produce voltage references v_d, v_q . Run at the FOC sample rate (in this implementation, every iteration of *loopFOC()*, typically 1.0 to 1.5 kHz on the UNO).
- Velocity loop (PI on ω): consumes the velocity setpoint and produces an i_q reference. Saturates against the current limit. Runs at the same sample rate as the inner loops.
- Position loop (P on θ): consumes the position setpoint and produces a velocity reference. Saturates against the velocity limit.

The cascade is parameterized by the control mode: in TORQUE mode, the outer loops are bypassed, and the user-supplied target is interpreted as i_q (current mode) or v_q (voltage fallback); in VELOCITY mode only the position loop is bypassed; in ANGLE mode the full cascade is active; in IMPEDANCE mode the outer loops are replaced by the spring-damper law of Section 2.6.

2.5.2. Cross-Axis Decoupling Feed-Forward

With knowledge of L_d, L_q and λ_{PM} , the cross-coupling and back-EMF terms in the dq model of Section 2.2.3 can be cancelled by adding compensation terms to the PI outputs:

$$v_{dtotal} = v_{dpi} + (-\omega_e \cdot L_q \cdot i_q)$$

$$v_{qtotal} = v_{qpi} + (\omega_e \cdot (L_d \cdot i_d + \lambda_{PM}))$$

The PI controllers then see a true single-axis first-order plant ($v = R \cdot i + L \cdot \frac{di}{dt}$) on each axis independently, dramatically improving the dynamic response in current mode, especially during fast acceleration or load step transients. Decoupling is an opt-in feature in MyFOC -it requires identified parameters - and is gated by the *decoupling_enabled* flag on the motor object.

2.5.3. Internal-Model-Control (IMC) Tuning of the Current Loops

Once decoupling has reduced each axis to the canonical first-order plant $G(s) = \frac{1}{L \cdot s + R}$, the textbook IMC tuning rule [7] for a desired closed-loop bandwidth ω_{bw} yields the PI gains:

$$K_p = L \cdot \omega_{bw}$$

$$K_i = R \cdot \omega_{bw}$$

Substituting and solving the closed-loop transfer function confirms that the resulting current-loop response is a first-order lag with time constant $\frac{1}{\omega_{bw}}$. The bandwidth ω_{bw} must be chosen below the PWM half-frequency (here $31.4 \text{ kHz} / 2 \approx 99 \text{ krad/s}$) and well below any sensor or controller delay; for the UNO platform a bandwidth of 800 rad/s ($\approx 127 \text{ Hz}$) was found to be a good engineering compromise - fast enough to reject load disturbances on a rehabilitation timescale, slow enough to be insensitive to the finite ADC and floating-point latency.

The MyFOC identifier module computes both gains automatically from the measured R and L_d (or L_q) once identification is complete, so the user need only specify the desired bandwidth.

2.5.4. Anti-Windup and Saturation

Every cascaded loop has a saturation limit downstream of its output: the current loop is bounded by $v_q \leq v_{limit}$; the velocity loop is bounded by $i_{q_{ref}} \leq i_{limit}$ when current sensing is enabled, or by $v_q \leq v_{limit}$ when voltage-mode fallback is active; the position loop is bounded by v_{limit} ; and the impedance controller is bounded by $torque_{limit}$. Whenever an output is clamped, integrator wind-up degrades the controller's recovery from saturation.

MyFOC implements a simple but effective back-calculation form of anti-windup: at each PI update, if the computed output exceeds the limit, the integrator is only updated in the direction that reduces saturation; conversely, it is held when its update would deepen saturation. The same logic is applied in both directions to handle bipolar setpoints.

A subtle requirement that surfaced during validation is that the PI's internal limit must agree with the physical limit at its output. When current sensing is enabled, the velocity PI's output is a current reference in amperes, and its internal limit must be set to i_{limit} rather than to the voltage limit; otherwise, the integrator winds up to many times the allowable current before its external clamp kicks in.

2.6. Impedance Control

Impedance control, introduced by Hogan in 1985 [8], treats the controlled actuator as a programmable mechanical impedance $Z(s) = \frac{F(s)}{V(s)}$ (or, equivalently in the rotational domain, $\frac{\tau(s)}{\omega(s)}$). Rather than enforcing a position or velocity reference and rejecting the operator's force as a disturbance, the controller deliberately yields to operator force in a manner specified by the chosen Z [9].

The simplest and most common parameterization is a linear second-order spring-damper, expressed as a torque law:

$$\tau = -K \cdot (\theta - \theta_{eq}) - B \cdot \omega$$

where K is the virtual rotational stiffness ($\text{N}\cdot\text{m}/\text{rad}$), B is the virtual rotational damping ($\text{N}\cdot\text{m}\cdot\text{s}/\text{rad}$) and θ_{eq} is the position at which the spring exerts no force. The corresponding closed-loop natural frequency and damping ratio, for a single-degree-of-freedom load with effective rotor inertia J_{eff} , are:

$$\omega_n = \sqrt{\frac{K}{J_{eff}}}$$

$$\zeta = \frac{B}{2 \cdot \sqrt{K \cdot J_{eff}}}$$

The clinical relevance is that K and B can be tuned independently of the underlying actuator dynamics - provided the current loop's bandwidth is well above ω_n and the actuator is back-drivable - to produce the rehabilitation modes specified in Section 6.4. The conversion from the desired torque τ to the q-axis current reference for the FOC inner loop is the standard $\tau = K_t \cdot i_q$ relationship:

$$i_{qref} = \frac{\tau}{K_t} = \frac{\tau}{\left(\left(\frac{3}{2}\right) \cdot pp \cdot \lambda_{PM}\right)}$$

This conversion is the principal reason that λ_{PM} must be identified before impedance mode can function correctly. The MyFOC impedance controller refuses to produce nonzero output if $K_t < 10^{-6}$, returning the motor to an idle state until parameters are populated.

2.7. SimpleFOC as a Reference Implementation

SimpleFOC [11] is an open-source Arduino-compatible FOC library that supports a wide range of microcontrollers and gate-driver topologies including the DRV8302 [2]. Its implementation is C++ with template-style abstraction over driver and sensor types, and its FOC pipeline follows the same conceptual structure adopted in this project: sensor $\rightarrow$ electrical angle $\rightarrow$ optional current sensing $\rightarrow$ Park transform $\rightarrow$ PI control $\rightarrow$ inverse Park $\rightarrow$ SVPWM $\rightarrow$ inverter. SimpleFOC is widely deployed, well documented and benchmarked against industrial implementations; for the purposes of this project, it serves as a known-good comparison reference rather than as a substrate to be modified.

MyFOC deliberately matches SimpleFOC's algorithmic choices wherever a design decision has a defensible alternative - amplitude-invariant Clarke convention, min-max SVPWM, voltage-mode default with optional current-loop upgrade, and the SimpleFOC-style sensor alignment using a quadrature voltage drive - so that any empirically-observed differences can be attributed to micro-architectural or numerical implementation rather than fundamental algorithmic divergence.

3. System Design and Architecture

3.1. Hardware Overview

The target platform was selected to meet three constraints: it must be inexpensive and widely available to undergraduate laboratories; it must include a gate driver suitable for a small three-phase BLDC motor with onboard current sensing; and it must allow the firmware to be ported to higher-performance microcontrollers with minimal effort. The chosen configuration is the ATmega328P [1] with the DRV8302 gate driver [2] and an AMT103 incremental encoder [10], summarized in Table 1.

The actuator itself was selected for the specific demands of impedance rendering rather than for raw output. A device that emulates a virtual spring–damper must produce smooth, accurately controlled torque at very low and at zero speed — an isometric hold is a stall condition — must be freely back-drivable so that the patient can always overpower it, and must be safe in direct contact with a limb. The IPOWER GM4108-120T is a high-pole-count outrunner surface-PMSM of the type developed to stabilise camera platforms, and it meets these demands directly. Its eleven pole pairs (twenty-two rotor magnets) give eleven electrical cycles per mechanical revolution, so the commutation torque is distributed finely around the rotor and the residual cogging and torque ripple that would otherwise corrupt a rendered spring are small. The motor is wound with many turns of fine wire — the 120T designation — which yields a high phase resistance ($11.16\ \Omega$) and a high torque constant ($0.570\ \text{N}\cdot\text{m}/\text{A}$) at a correspondingly low velocity constant: it produces useful torque at a fraction of an ampere and at standstill rather than requiring high current or high speed. This is the opposite of the optimisation used in the brushless motors sold for drones and radio-controlled vehicles, which are wound for a high velocity constant so as to spin propellers and wheels at tens of thousands of revolutions per minute; those motors are thermally stressed at stall and coarse in their torque control near zero speed, which is exactly the regime a haptic device operates in.

Two further properties reinforce the choice. First, the motor uses direct-drive, with the lever fixed straight to the rotor and no reduction gearing. A gearbox would multiply torque but introduce backlash, added friction and reflected inertia, all of which degrade transparency and make the device feel notchy or sticky rather than rendering the commanded impedance cleanly; direct drive preserves the low-friction back-drivability on which impedance control depends, and the transparency (back-drive) test of Section 6.5 is only meaningful because the actuator is direct-drive. Second, a surface-mounted-magnet PMSM of this kind has a near-sinusoidal back-EMF, which matches the sinusoidal field-oriented commutation used throughout this work; a trapezoidal-back-EMF BLDC motor driven with sinusoidal FOC would exhibit additional torque ripple. The high-torque-constant, low-current operating point also keeps the electrical power, and therefore the energy that could be delivered to a limb, low: a small gate driver and a 0.6 A current limit suffice, which is appropriate for a device intended for human contact.

Smoothness at this level is not unique to small gimbal motors. The same design principles — a high pole count, a low-cogging slot-and-magnet geometry and direct drive — are used in the frameless direct-drive torque motors found in robot joints, telescope mounts and collaborative-robot actuators, which deliver the same low-ripple behaviour at far higher

torque; a gimbal motor is essentially a small, inexpensive instance of that class. Those larger machines, however, demand proportionally larger drive currents, bigger gate drivers and more careful thermal design, and typically cost one to two orders of magnitude more, none of which suits an graduate-accessible platform. The gimbal motor's limitations are the corollary of its strengths: the fine windings that give it a high torque constant also cap its continuous current, and hence its continuous torque and power, and its usable speed range is modest. For a benchtop trainer working against light loads — the 0.30 N·m ceiling and 200 g deadweight of Section 6.5 — these limits are not binding, but a device required to deliver larger forces or higher speeds would move to a frameless torque motor of the kind just described.

Table 1: Hardware configuration.

Component	Specification
Microcontroller	Atmel/Microchip ATmega328P (Arduino UNO Rev3)
Clock frequency	16 MHz (external crystal)
Program memory	32 KB Flash (29 KB available after bootloader)
Data memory	2 KB SRAM
EEPROM	1 KB
Gate driver	Texas Instruments DRV8302 in 3-PWM mode
DC bus voltage	12 V
Current-sense gain	10× (default GAIN strap), shunt 5 mΩ → 0.05 V/A
Motor	IPOWER GM4108-120T 11-pole-pair surface-PMSM gimbal motor
Position sensor	CUI AMT103 incremental quadrature encoder, configured at 2048 PPR (8192 CPR after 4× decoding)
DRV protection signals	nFAULT (fault), nOCTW (over-current warning), open-drain active-LOW

All three high-side gate inputs (INH_A , INH_B , INH_C) are driven from Arduino pins 9, 10 and 11 respectively. The DRV8302's internal current-sense amplifier outputs SO1 and SO2 are routed to analog inputs A2 and A3; this routing is constrained by the user's wiring of the protection signals nOCTW and nFAULT to A0 and A1. The driver enable input EN_{GATE} is on pin 8. The three configuration strap pins M_{OC} , M_{PWM} and OC_{ADJ} are tied via Arduino pins 5, 6 and 7 to LOW, HIGH and HIGH respectively, selecting 3-PWM mode with the hardware over-current protection active and the over-current threshold at its maximum factory setting. Pin assignments are tabulated in Table 2.

Table 2: Pin assignments.

Signal	Pin
INH_A (high-side PWM A)	Arduino digital pin 9 (Timer1 OC1A)

Signal	Pin
INH_B (high-side PWM B)	Arduino digital pin 10 (Timer1 OC1B)
INH_C (high-side PWM C)	Arduino digital pin 11 (Timer2 OC2A)
EN_{GATE}	Arduino digital pin 8
M_{PWM} (mode selector)	Arduino digital pin 6 $\leftarrow$ HIGH (3-PWM mode)
M_{OC} (OC enable)	Arduino digital pin 5 $\leftarrow$ LOW (OC protection on)
OC_{ADJ} (OC threshold)	Arduino digital pin 7 $\leftarrow$ HIGH (maximum threshold)
SO1 (CS amp A)	Arduino analog A2
SO2 (CS amp B)	Arduino analog A3
nOCTW (over-current warning)	Arduino analog A0 (used as digital input with pull-up)
nFAULT (fault output)	Arduino analog A1 (used as digital input with pull-up)
Encoder A	Arduino digital pin 2 (INT0)
Encoder B	Arduino digital pin 3 (INT1)

3.2. Software Architecture

The firmware is organised as a single Arduino library, MyFOC, with one C++ namespace MyFOC enclosing all classes and free functions. Each library module has a single responsibility, and the public interface is small enough so that the entire library can be visualized at once. Modules are listed in Table 3.

Table 3: Module decomposition of the MyFOC library.

Module	Kind	Responsibility
MyFOCMath	header + impl	Mathematical kernel: angle normalisation, fast sin/cos via 65-entry quarter-wave LUT in PROGMEM, Clarke, Park, inverse Park, inverse Clarke, sinusoidal PWM and min-max SVPWM.
MyPID	header + impl	Discrete-time PID controller with trapezoidal integration, adaptive sample time (micros()),

Module	Kind	Responsibility
		output saturation, back-calculation anti-windup, and optional output ramp limiting.
MyLPF	header + impl	First-order low-pass filter with adaptive sample time and tunable time constant T_f .
MyEncoder	header + impl	Quadrature encoder driver using CHANGE-mode external interrupts on pins 2/3. 4× decoding; signed long position counter; instantaneous velocity from finite-differences in radians per second.
MyBLDCDriver3PWM	header + impl	Three-phase 3-PWM inverter driver. Configures Timer1 + Timer2 for 31.4 kHz phase-correct PWM and writes OCR registers directly.
MyCurrentSense3PWM	header + impl	Two-channel DRV8302 current-sense reader. Boots the AVR ADC clock to 1 MHz; calibrates per-channel zero offsets; reconstructs phase C from $i_a + i_b + i_c = 0$.
MyBLDCMotor	header + impl	Top-level motor object: sensor-alignment, pole-pair check, all four control modes, inner current loop with optional decoupling, voltage-mode fallback, CSV monitoring sink.
MyMotorID	header + impl	Motor-parameter identification (MyMotorIdentifier class): open-loop resistance at a parked d-axis, d-axis inductance from a step response, flux linkage from a back-EMF spin, and IMC current-loop auto-tuning from the identified R and L.
MyFOC.h	umbrella	Single-included header that pulls in all of the above.

The total source size is approximately 2,500 lines of C++ across 17 files (8 headers, 8 source files, 1 umbrella). The compiled footprint on AVR is about 21 KB of program memory and zero static SRAM beyond per-object instance state, leaving over 10 KB of flash and the bulk of SRAM for user-application code.

Rehab_Device.ino - Application: impedance control with runtime-tunable K , B and θ_{eq} , a velocity-gated low-pass-filtered adaptive scaling of stiffness, damping and ceiling, hardware

fault monitoring on nFAULT and nOCTW, 50 Hz CSV logging with per-procedure T<n> tagging.

3.3. Control-System Architecture

The inner loop *loopFOC()* runs as fast as the main loop will allow, typically every 700 μ s to 1 ms depending on user code overhead; the outer loop *move()* runs at the same rate and consumes the latest sensor readings. Each loop iteration:

- Reads the current encoder count atomically (ISRs may have updated it since the previous loop) and converts to *shaft_angle* in radians.
- Computes the electrical angle via $\theta_e = \text{dir} \cdot \text{pp} \cdot \text{shaft_angle} - \text{zero_elec_angle}$, with $\text{dir} \in \{+1, -1\}$ encoding the empirically determined direction sense established during alignment.
- Reads the two-phase currents from ADC, reconstructs i_c , applies the forward Clarke and Park transforms to obtain (i_d, i_q) in the rotor frame.
- Computes the q-axis error $\text{err}_q = i_{q_target} - i_q$ and runs the PI_{iq} controller to produce $v_{q\pi}$; similarly produces $v_{d\pi}$ from the d-axis error (which has target 0 for non-field-weakening operation).
- Adds the cross-axis decoupling feed-forward terms (Section 2.5.2) if *decoupling_enabled* is set and parameters have been identified.
- Saturates v_d and v_q against *voltage_limit*, calls the inverse Park, inverse Clarke and min-max SVPWM, and writes the resulting duty cycles to the Timer1/Timer2 OCR registers.

The outer loop runs after the inner one. Depending on motor.controller it: (TORQUE) sets *i_q_target* directly from motor.target; (VELOCITY) runs *PI_velocity* on (*target-shaft_velocity*) and saturates the output to *current_limit* before assigning to *i_q_target*; (ANGLE) runs *P_angle* on the position error to produce a velocity setpoint, saturates against *velocity_limit*, then runs *PI_velocity* to produce *i_q_target*; (IMPEDANCE) computes $\tau_{des} = -K(\theta - \theta_{eq}) - B \omega$, saturates against *torque_limit*, and converts to *i_q_target* via the torque constant K_t .

When the current sensor is not linked (voltage-mode fallback), all of the above paths instead write directly into *voltage_q*, and *loopFOC()* bypasses the current loop and Park-based feedback: it computes θ_e and immediately calls *setPhaseVoltage(voltage_q, 0, θ_e)*. This degraded mode reproduces the classical SimpleFOC voltage-mode behaviour and is retained as a usability fallback in environments where the current-sense routing is unavailable.

3.4. Safety Architecture

Multiple independent layers of saturation guard against unsafe operating conditions:

- Inverter duty saturation: each PWM duty cycle is clamped to [0, 1] inside *setDutyCycle()* so an out-of-bound v_α , v_β cannot produce a register overflow.
- Voltage saturation: $|v_d|, |v_q| \leq \text{voltage_limit}$ (default 12 V; 8 V in the rehab demo).
- Current saturation: $|i_{q_ref}| \leq \text{current_limit}$ (default 2.0 A; 0.6 A in the rehab demo).

- Torque saturation: in impedance mode, $|\tau_{des}| \leq \text{torque_limit}$ (set to 0.08 N·m for the rehab demo, a clinically conservative value for a small gimbal motor).
- Velocity saturation: in angle mode, $|\omega_{setpt}| \leq \text{velocity_limit}$; *PID_velocity.limit* is also dynamically reassigned (*current_limit* in current mode, *voltage_limit* in voltage mode) so that anti-windup operates at the correct level.
- Hardware over-current protection: the DRV8302's internal OC comparator latches nFAULT low if any half-bridge sees current beyond the threshold programmed by OC_ADJ; the rehab sketch monitors this signal and calls *driver.disable()* within one main-loop iteration when it asserts.
- Soft over-current warning: nOCTW asserts low when current exceeds a warning threshold below the latched OC trip; the sketch prints a serial warning so that an operator can reduce torque demand.

This layered structure satisfies the engineering principle that no single fault - software, firmware or hardware - should be able to produce a torque exceeding the configured ceiling. The rehabilitation device uses a clinically conservative 0.08 N·m default. In a clinical deployment of the rehabilitation application additional layers would be required (e.g. an independent watchdog and a mechanical torque limiter), but the present scope is the firmware layer.

4. Implementation

4.1. Mathematical Kernel (MyFOCMath)

4.1.1. Sin/Cos Lookup Table

The ATmega328P does not have a hardware floating-point unit. The standard *math.h* *sinf()* and *cosf()* functions are implemented in software using polynomial approximation and take approximately 120 μ s per call on the 16 MHz core. For a FOC inner loop in which sin and cos are needed twice per iteration (once in Park, once in inverse Park) and a 1 kHz loop rate, that overhead alone consumes nearly 50 % of the available CPU. A lookup-table approximation is therefore necessary.

MyFOCMath provides a 65-entry table covering $[0, \frac{\pi}{2}]$ in 1.4° steps, stored in PROGMEM to avoid consuming the limited 2 KB SRAM. Quarter-wave symmetry extends the table to the full $[0, 2\pi)$ domain:

- Q1, $0 \leq \theta < \pi/2$: $\sin \theta = \text{table}[\text{idx}]$
- Q2, $\pi/2 \leq \theta < \pi$: $\sin \theta = \text{table}[64 - (\text{idx from } \pi/2)]$
- Q3, $\pi \leq \theta < 3\pi/2$: $\sin \theta = -\text{table}[\text{idx from } \pi]$
- Q4, $3\pi/2 \leq \theta < 2\pi$: $\sin \theta = -\text{table}[64 - (\text{idx from } 3\pi/2)]$

Linear interpolation between table entries yields an error below 1×10^{-4} across the full domain, well below the precision available from the 10-bit ADC feedback. The single-call *sinCos*(θ , *s*, *c*) is provided as a convenience for the two trigonometric values needed at every Park step. Each LUT call is approximately 9 μ s, a 13× speedup over the *math.h* implementation.

4.1.2. Clarke, Park, Inverse Park, Inverse Clarke

The four core transformations are implemented as inline functions and follow the conventions established in Section 2.2. The amplitude-invariant Clarke is used throughout. For surface-PMSM and balanced-phase operation the inverse Clarke assumes $v_a + v_b + v_c = 0$, which holds for any output of the min–max SVPWM modulator since the modulator subtracts the common-mode component. Each transform compiles to roughly 40 instructions on AVR and executes in under 15 μs .

4.1.3. Space-Vector PWM

The min–max injection from Section 2.3.2 is implemented as a single function:

```
inline void svpwm(float va, float vb, float vc, float v_dc,
                 float& duty_a, float& duty_b, float& duty_c) {
    float vmax = va;
    float vmin = va;
    if (vb > vmax) vmax = vb;
    if (vc > vmax) vmax = vc;
    if (vb < vmin) vmin = vb;
    if (vc < vmin) vmin = vc;
    float voff = 0.5f * (vmax + vmin);
    duty_a = (va - voff) / v_dc + 0.5f;
    duty_b = (vb - voff) / v_dc + 0.5f;
    duty_c = (vc - voff) / v_dc + 0.5f;
    // saturation to [0, 1] follows
}
```

The output duty cycles are clamped to $[0, 1]$ to protect against numerical overshoot. Total execution time on AVR is approximately 25 μs .

4.2. PWM Driver (MyBLDCDriver3PWM)

The driver dedicates two of the ATmega328P's three timers [1] to PWM generation. Timer 0 is reserved for the Arduino core's *millis()/delay()* services, so Timer 1 (16-bit) and Timer 2 (8-bit) carry the three PWM outputs:

- Timer 1: WGM[3:0] = 0001 (Mode 1, 8-bit phase-correct PWM, TOP = 0xFF). COM1A = COM1B = non-inverting. CS = 001 (no prescaler). Outputs OC1A on Arduino pin 9 and OC1B on pin 10.
- Timer 2: WGM[2:0] = 001 (Mode 1, phase-correct PWM, TOP = 0xFF). COM2A = non-inverting. CS = 001 (no prescaler). Output OC2A on pin 11.

Phase-correct PWM is preferred over fast PWM for motor-control applications because the two timers remain edge-synchronised at the centre of every PWM cycle, which minimises current-ripple beat patterns and acoustic noise. The resulting carrier frequency is:

$$f_{PWM} = \frac{F_{CPU}}{2 \cdot N \cdot TOP} = 16 \times \frac{10^6}{2 \cdot 1 \cdot 255} \approx 31\,372\text{ Hz}$$

comfortably above the 20 kHz audible threshold and well within the DRV8302's slew-rate envelope. Duty cycles supplied by the FOC pipeline are converted to 8-bit OCR values via $OCRxA = \text{uint8_t}(duty \cdot 255)$. The *init()* routine also leaves the enable pin LOW so that the gate driver cannot be active inadvertently during boot; the *enable()* method asserts it once the FOC software is ready.

4.3. Quadrature Encoder Decoder (MyEncoder)

CHANGE-mode interrupts on pins 2 (INT0) and 3 (INT1) trigger on every edge of channels A and B respectively, giving 4× decoding. Inside each ISR both channel states are read, and a 32-bit signed pulse counter is updated. The direction-decoding logic is derived as follows. After any edge in the canonical forward sequence (A,B) = 00 → 10 → 11 → 01 → 00 - and only that sequence - the post-edge state of (A,B) for an A-channel transition is either (1, 0) or (0, 1); in both cases $A \neq B$. For the reverse sequence the corresponding post-edge states are (0, 0) and (1, 1); in both cases $A = B$. The dual analysis applies to B-channel transitions. The full logic table is:

Table 4: Quadrature decoding logic.

Edge	Post-edge (A,B)	Direction	Decoder rule
A rising	10	Forward	$A \neq B \rightarrow +1$
A falling	01	Forward	$A \neq B \rightarrow +1$
B rising	11	Forward	$A = B \rightarrow +1$
B falling	00	Forward	$A = B \rightarrow +1$
A rising	11	Reverse	$A = B \rightarrow -1$
A falling	00	Reverse	$A = B \rightarrow -1$
B rising	01	Reverse	$A \neq B \rightarrow -1$
B falling	10	Reverse	$A \neq B \rightarrow -1$

Reading a 32-bit long counter from the main loop while interrupts may update it requires a short critical section to avoid a torn read; this is implemented as *noInterrupts()/interrupts()* bracketing a one-line load. Velocity is computed from the finite difference of the counter divided by the elapsed time in *micros()* between calls. The raw velocity is noisy; an MyLPF instance with $T_f = 10$ ms is used in the motor object to smooth it before feeding the velocity PI.

4.4. Current Sensing (MyCurrentSense3PWM)

The DRV8302's two internal current-sense amplifiers are biased to $\frac{V_{CC}}{2}$ and have a transconductance set by the GAIN strap [2]. With the default 10× gain and the 5 mΩ shunts on the reference breakout, the sensitivity is 0.05 V/A; the ADC step at $V_{ref} = 5$ V is 4.88 mV, giving a digital resolution of 10.24 counts per ampere. Three implementation details are worth highlighting.

4.4.1. Zero calibration

The amplifier output is biased to $\frac{V_{CC}}{2} \approx 1.65\text{ V}$ at zero current - here V_{CC} is the sense amplifiers' 3.3 V analog reference, and the as-built bench confirmed this bias as roughly 344 ADC counts (Section 6.2). Because it also depends on the breakout's SLEEP/EN_GATE wiring and on amplifier and resistor tolerances, the offset is measured rather than assumed. *calibrateZero()* averages 300 ADC readings ($\approx 200\text{ }\mu\text{s}$ each at the boosted ADC clock) per channel with the driver enabled at 50% duty - which produces zero net phase voltage and hence zero current - and stores the result as *offset_A_counts*, *offset_B_counts*. Subsequent current measurements subtract this offset before scaling.

4.4.2. Reconstruction of i_c

Phase C is not sensed directly. For a star-connected motor with a floating neutral the sum $i_a + i_b + i_c = 0$ holds at all times (no charge can accumulate in the neutral node), so $i_c = -i_a - i_b$. This relationship is for the balanced case and is used throughout the firmware.

4.4.3. ADC clock acceleration

The default Arduino ADC prescaler is 128, giving a 125 kHz ADC clock and approximately 104 μs per 10-bit conversion. For the inductance step-response test (Section 4.8.2), which must resolve a time constant $\frac{L}{R}$ on the order of 40 μs , this is far too slow. The *MyCurrentSense3PWM init()* method reprograms the ADCSRA prescaler bits to 100 (prescaler = 16), producing a 1 MHz ADC clock and approximately 13 μs per conversion:

```
// ADCSRA = ADEN ADSC ADATE ADIF ADIE ADPS2 ADPS1 ADPS0
// Set ADPS[2:0] = 100 → prescaler 16 → 1 MHz ADC clock
ADCSRA = (ADCSRA & 0xF8) | 0x04;
```

This rate is above the ADC's nominal 200 kHz maximum-for-full-accuracy specification, but the empirical accuracy loss is approximately 1 LSB, which is well below the PWM-induced current ripple and is in any case averaged out by the PI inner loop and by the offset-calibration averaging.

4.5. PID and Low-Pass Filter (MyPID, MyLPF)

Both controllers measure their own sample time using *micros()* between successive invocations, so they automatically adapt to any jitter in the main loop. The PID equation discretisation is:

$$e[k] = \text{setpoint} - \text{measurement}$$

$$\text{integral} \leftarrow \text{integral} + K_i \cdot T_s \cdot 0.5 \cdot (e[k] + e[k-1]), \quad (\text{trapezoidal})$$

$$\text{derivative} = K_d \cdot \frac{e[k] - e[k-1]}{T_s}$$

$$u[k] = K_p \cdot e[k] + \text{integral} + \text{derivative}$$

If $u[k]$ exceeds the configured $\pm\text{limit}$, the integrator is only updated in the direction that reduces saturation (back-calculation anti-windup). Optional output ramp limiting, bounded by $\text{ramp} \cdot T_s$ per step, is provided for fragile setpoints.

MyLPF implements the canonical first-order filter:

$$y[k] = \alpha \cdot y[k-1] + (1 - \alpha) \cdot x[k], \quad \text{with } \alpha = \frac{T_f}{T_f + T_s}$$

Special-case $T_f \leq 0$ disables the filter (pass-through), simplifying the user's life when LPF behaviour must be turned off mid-experiment.

4.6. Motor Object and FOC Pipeline (MyBLDCMotor)

The motor class encapsulates the entire FOC state machine. Public members expose pole-pair count, the four identified parameters, every safety limit, control mode, target, all four control-loop instances, the impedance parameters and the runtime state (angles, currents, velocity). Pointer members link to the encoder, driver, optional current sensor and optional monitoring stream. The principal methods are *init()*, *initFOC()*, *loopFOC()*, *move()* and *setPhaseVoltage()*.

Inside *loopFOC()*, if a current sensor is attached, *readCurrents()* populates the *abc*, *αβ* and *dq* current arrays; *applyCurrentLoop()* runs *PI_id* and *PI_iq*, adds the optional decoupling, saturates against *voltage_limit*, and stores the results in *voltage_d* and *voltage_q*. *setPhaseVoltage(voltage_q, voltage_d, electrical_angle)* performs the inverse Park, inverse Clarke and SVPWM in that order and writes the PWM duty cycles to the driver. The total inner-loop execution time is approximately 700 μs on the UNO, dominated by the four trigonometric LUT calls (two in Park, two in inverse Park) and the two ADC conversions.

4.7. Sensor Alignment (alignSensor)

The alignment procedure determines two unknowns: the direction sense of the encoder (whether positive counter increments correspond to forward or reverse rotation in the convention assumed by the FOC math), and the offset *zero_elec_angle* that maps the encoder reading to the electrical angle of the rotor d-axis.

The procedure consists of four phases:

1. Sweep the electrical angle from 0 to 2π in 200 steps with $v_q = U_{align}$ applied and $v_d = 0$. The rotor follows the rotating stator MMF and completes one full electrical revolution, ending at electrical angle 0 ($= 2\pi$ modulo).
2. Read the mechanical angle $\theta_m, zero$.
3. Sweep the electrical angle from 0 to π in 100 steps. Read θ_m, π .
4. Sweep back from π to 0. Read $\theta_m, zero$.

Direction is determined from $sign(\theta_m, \pi - \theta_m, 0)$: if positive, *sensor_direction* = CW; if negative, CCW; if essentially zero, alignment failed and the rotor did not move (likely insufficient drive voltage or a stalled mechanical load).

Computing the electrical-angle offset *zero_elec_angle* requires careful geometric reasoning. At the end of step 4, the last commanded value was $v_q = U_{align}$ with electrical angle 0. By the geometric content of the Park transform (Section 2.2.2), applying v_q at electrical angle 0 produces a stator MMF vector at angle $\frac{\pi}{2}$ in the *αβ* plane. The rotor's permanent magnet d-axis settles aligned with this MMF, so the rotor's true electrical angle is $\frac{\pi}{2}$, not 0. The offset must therefore be chosen so that *computeElectricalAngle()* returns $\frac{\pi}{2}$ (matching physical reality) at the mechanical angle $\theta_m, zero$:

$$\theta_e = \text{dir} \cdot \text{pp} \cdot \theta_m - \text{zero_elec_angle}$$

$$\text{zero_elec_angle} = \text{dir} \cdot \text{pp} \cdot \theta_{m, \text{zero}} - \frac{\pi}{2}$$

This $\frac{\pi}{2}$ correction is the source of a subtle defect in earlier drafts of the firmware: if the offset is set to $\text{dir} \cdot \text{pp} \cdot \theta_{m, \text{zero}}$ (without the $-\frac{\pi}{2}$ term), then commanded v_q produces no net torque because the FOC's claimed θ_e is aligned with the stator MMF rather than perpendicular to it.

4.8. Motor Parameter Identification

4.8.1. Resistance

With the rotor open-loop-parked at electrical angle 0 by applying $v_d = U_{\text{test}}$ and ramping U over 100 steps to avoid current overshoot, the steady-state d-axis equation $v_d = R \cdot i_d + \omega_e \cdot L_q \cdot i_q$ reduces (since $\omega_e = 0$ and $i_q = 0$) to $v_d = R \cdot i_d$. After 500 ms of settling, the algorithm averages 200 samples of i_d (computed at the known electrical angle $\theta_e = 0$, not at the FOC's reading, so the measurement is decoupled from any residual alignment error) and reports:

$$R_{\text{phase}} = U_{\text{test}} / i_{d, \text{avg}}$$

The $U_{\text{test}} = 2$ V default produces approximately 0.4 A on a typical 5 Ω gimbal motor, yielding an ADC deviation of about four counts. With 200 averages the noise contribution drops by a factor of $\sqrt{200} \approx 14$, yielding a measurement standard deviation in the milli-ohm range.

4.8.2. Inductance

The d-axis voltage equation in the rotor-stationary, no-back-EMF regime is $v_d = R \cdot i_d + L_d \cdot \frac{di_d}{dt}$. A step from 0 to U_{test} produces the first-order response:

$$i_d(t) = \left(\frac{U_{\text{test}}}{R}\right) \cdot (1 - \exp\left(-\frac{t}{\tau}\right)), \quad \tau = \frac{L_d}{R}$$

The algorithm parks the rotor with a low-bias $v_d = 0.3 \cdot U_{\text{test}}$, then steps abruptly to U_{test} and polls the ADC as fast as possible (one analogRead per loop iteration, $\approx 13 \mu\text{s}$ at the boosted ADC clock) until $|i_d|$ exceeds $0.632 \cdot (U_{\text{test}} / R)$. At that point the elapsed time equals τ , and:

$$L_d = R_{\text{phase}} \cdot \tau$$

With the rotor parked at electrical angle 0, the phase-A current equals i_α equals i_d (within a sign), so the algorithm reads phase A only (one ADC) rather than performing the full Clarke + Park (two ADCs plus arithmetic), halving the sample period. The surface-PMSM assumption $L_d \approx L_q$ is adopted; the q-axis can be re-measured by parking the rotor at $\theta_e = \frac{\pi}{2}$ and repeating the procedure, but for the present motor and application the single-axis measurement is sufficient.

4.8.3. Flux Linkage

Once R and L_d are populated, the only remaining electrical parameter needed to map q-axis current to torque is the permanent-magnet flux linkage λ_{PM} . `identifyFluxLinkage( $\omega_e$ )` measures it directly: the rotor is driven at a fixed low electrical speed ω_e under closed-loop velocity control and the steady q-axis voltage is recorded. In steady state the resistive and inductive drops are known, so the residual is the back-EMF E and $\lambda_{PM} = E / \omega_e$ - equivalently $\lambda_{PM} = (v_q - R \cdot i_q) / \omega_e$ in the surface-PMSM model with i_d regulated to zero.

The torque constant follows as $K_t = 1.5 \cdot p \cdot \lambda_{PM}$, where p is the number of pole pairs. For this motor the procedure returns $\lambda_{PM} \approx 0.0346$ Wb and $K_t \approx 0.570$ N·m/A (Table 6), closing the voltage-to-torque mapping $\tau = K_t \cdot i_q$. The complementary step of converting these constants into current-loop gains is the IMC tuning described in Section 2.5.3 and exercised by the identification sketch in Section 5.2.1.

4.9. Impedance Controller

Impedance mode replaces the outer control cascade with the spring-damper law of Section 2.6. At each call to `move()`, the controller computes:

```
pos_dev = shaft_angle - impedance_eq
tau_des = -impedance_K * pos_dev - impedance_B * shaft_velocity
saturate tau_des to ±torque_limit
Kt = (3/2) * pp * lambda_PM
if Kt < 1e-6:    park motor (refuse to operate without identified Kt)
elif current_sense linked:
    iq_ref = tau_des / Kt
    saturate iq_ref to ±current_limit
    current_q_target = iq_ref
else:           // voltage-mode approximation
    voltage_q = R_phase * (tau_des / Kt)
```

The voltage-mode branch produces only an approximation because it ignores the $\frac{di}{dt}$ and $\omega_e \cdot \lambda_{PM}$ terms; in practice the current-mode branch should always be used in a clinical or experimental setting. The graceful refusal to operate when K_t is unknown protects against accidentally driving the motor under unidentified parameters.

4.10. Data Logging and Monitoring

The motor object exposes `logCSVHeader()` and `logCSV()` methods that emit one comma-separated record per call with the columns: time-ms, target, shaft-angle, shaft-velocity, i_d , i_q , v_d , v_q and computed torque. In the rehab demo a 50 Hz logger writes this record to the serial port so that a Python script can capture and plot the rehabilitation session. The fixed schema simplifies offline analysis.

To efficiently test the setup, a motor mount was designed in Fusion 360 and 3D printed.

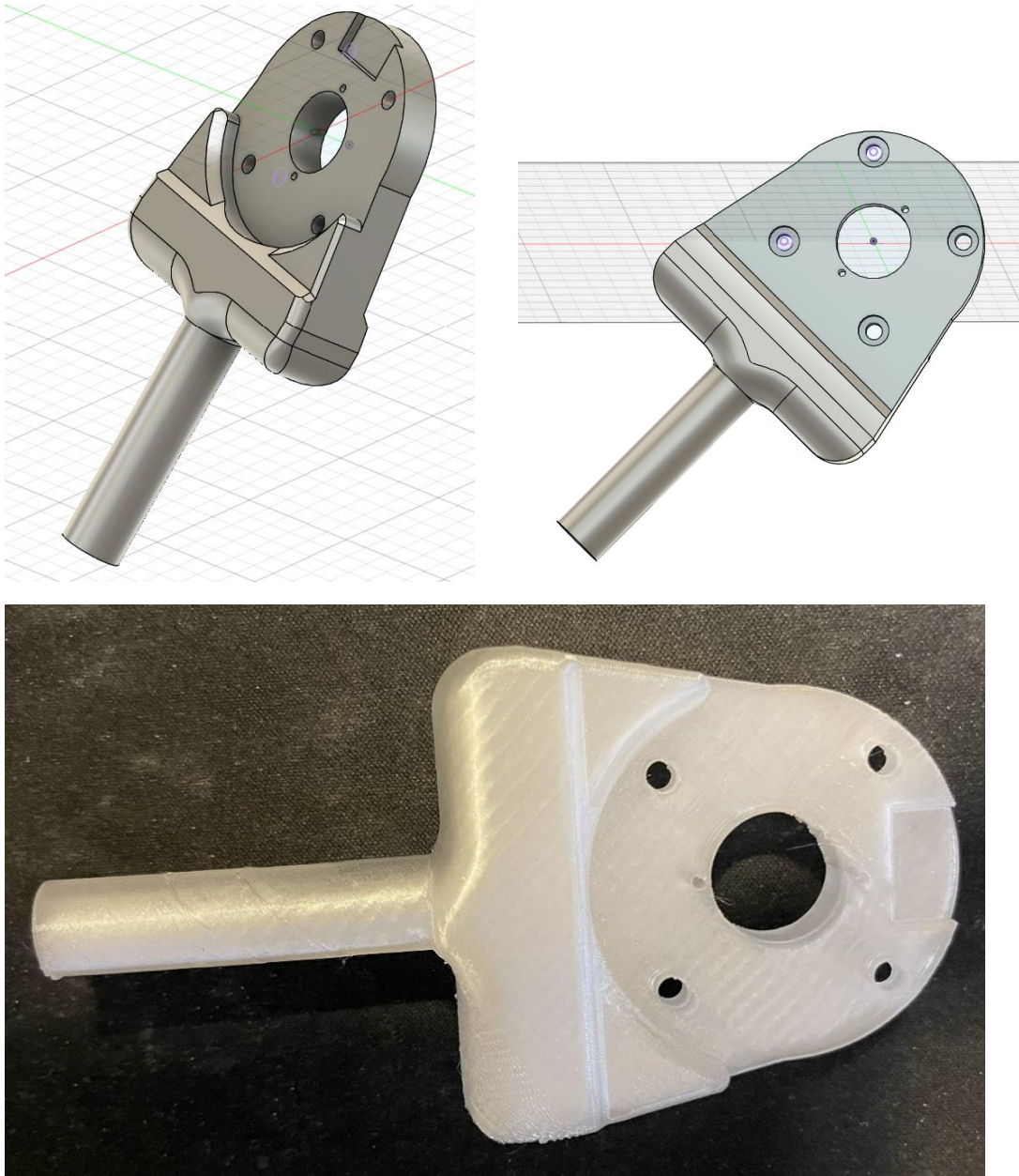


Figure 3: IPOWER GM4108-120T Mount 3D design.

The pendulum arm, which was also designed and 3D printed, will be mounted on the motor. The bottom part of the pendulum will serve to increase weight by adding scale calibration weight if needed. A custom piece was designed to connect the rotating gimbal with the AMT103 Encoder.

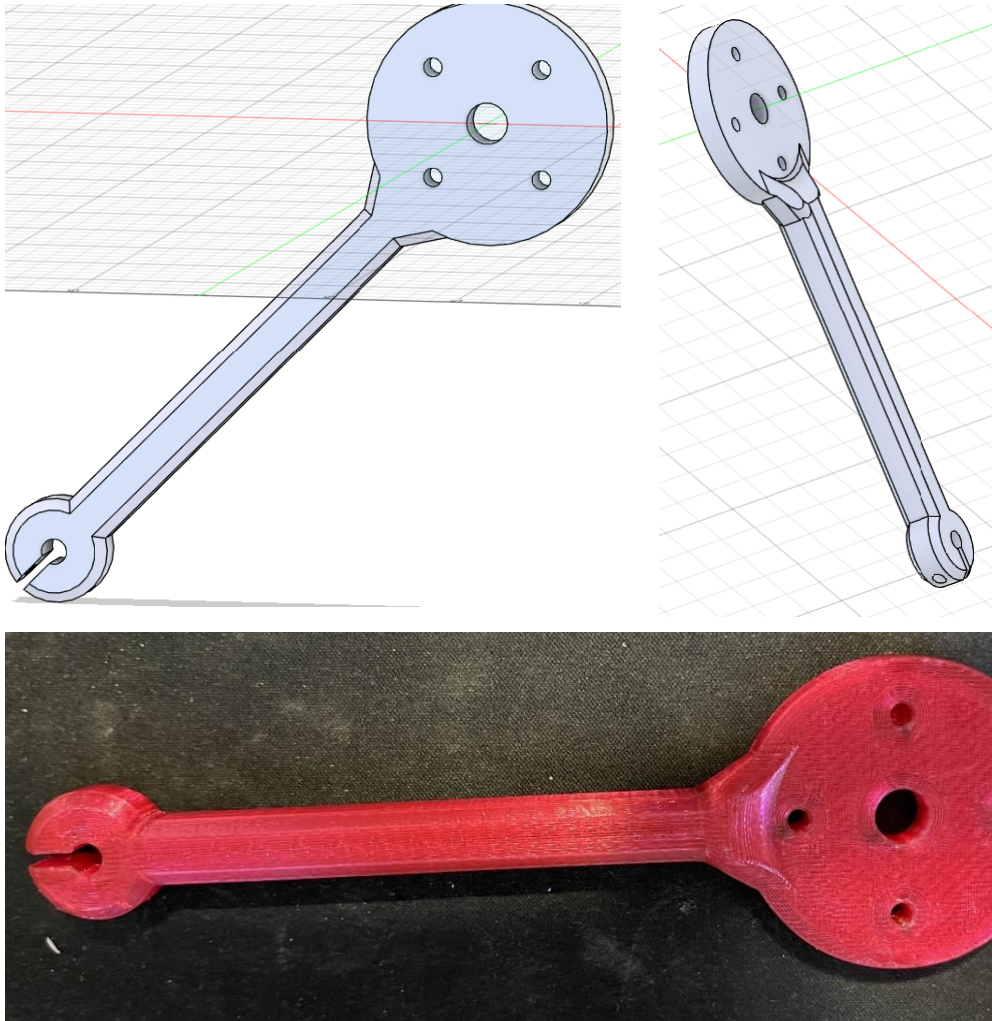


Figure 4: Pendulum Arm 3D design.

The final setup is as follows:

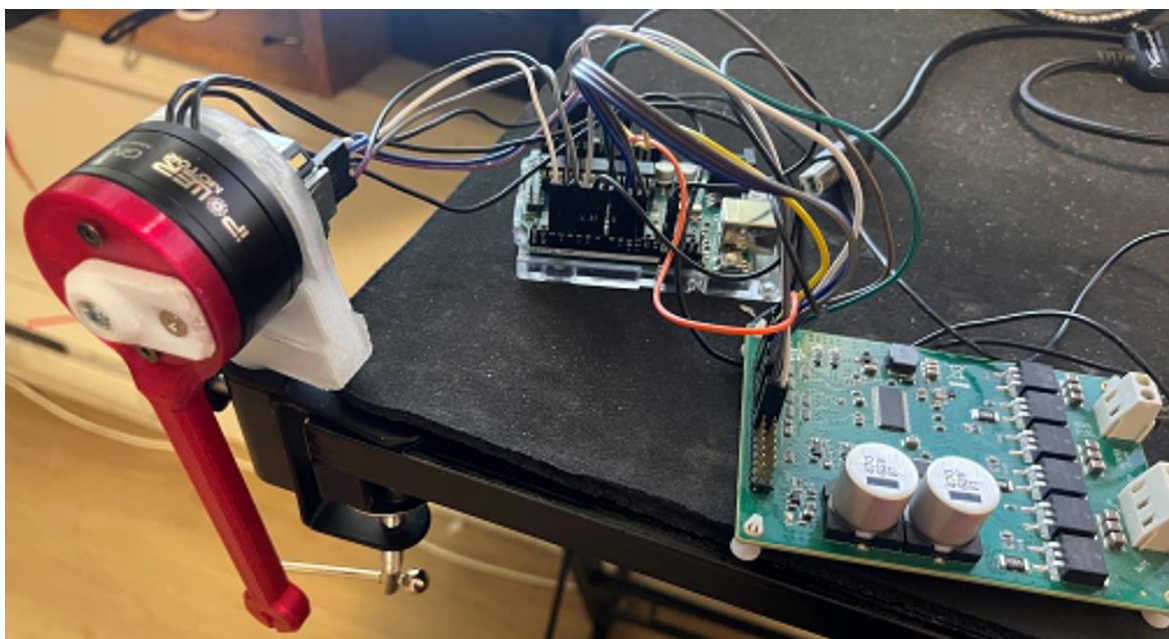


Figure 5: The final setup with everything connected.



Figure 6: Close up of the connected gimbal with encoder.

The 12 V DC supply is a bench power supply with a 5 A current limit (the actual value is usually around 2 A), set conservatively below the steady-state stall current of the motor under the strictest safety limit. Current limiting at the supply provides one further independent fault layer beyond the firmware and the DRV's own protection. In this particular case, a DC Power Bench and a Switched-Mode Power Supply (12V – 5A) are chosen for affordability.



Figure 7: The power benches used for this project.

All measurement is via the Arduino's built-in USB-serial bridge at 115200 baud, captured by a host laptop running a Python script (pyserial + pandas + matplotlib) that ingests the CSV stream emitted by *logCSV()* and produces the plots described in Section 6.

5.2. Test Procedures

Three structured test procedures are defined: parameter identification, open-loop FOC validation, and the clinical exercise-mode sequences used to drive the rehabilitation application.

5.2.1. Parameter Identification Procedure

The parameter-identification procedure was run with the motor mechanically unloaded but free to rotate. It:

1. Initialises the encoder and driver.
2. Calibrates the current-sense zero offsets with the driver disabled (300 samples per channel).
3. Enables the driver and runs *initFOC()* to align the sensor and verify the pole-pair count.
4. Pauses 500 ms to let the rotor settle.
5. Runs *identifyResistance()*, printing the measured R_{phase} .
6. Runs *identifyInductance()*, printing τ and L_d .
7. Runs *tuneCurrentLoops(800 rad/s)* to set the IMC gains.
8. Runs *identifyFluxLinkage(10 rad/s)*, spinning the motor briefly under closed-loop velocity control, printing λ_{PM} and K_t .
9. Prints a summary block and a copy-paste const-float block ready for the application sketch.
10. Disables the driver.

A successful identification yields all four parameters within the expected ranges for a small gimbal motor ($R \approx 1\text{--}15\ \Omega$, $L \approx 50\text{--}500\ \mu\text{H}$, $\lambda_{PM} \approx 0.005\text{--}0.05\ \text{Wb}$). Outlier values almost always indicate hardware miswiring or insufficient supply current, both of which are diagnosed by inspecting the per-step serial output.

5.2.2. FOC Validation Procedure

A minimal voltage-mode FOC sketch, developed during bring-up for direct comparison with SimpleFOC, exposed the M / C / S serial command set; the following sequence was executed:

1. C0 (torque mode); M1.0 (1 A target). Observe that the motor begins to spin smoothly and accelerates until the velocity-dependent friction torque equals $K_t \cdot 1\ \text{A}$.
2. C1 (velocity mode); M5.0 (5 rad/s setpoint). Observe steady-state velocity tracking. Apply a manual load on the shaft; observe that the controller responds by raising i_q to maintain velocity.
3. C2 (angle mode); M0 (zero-angle setpoint). Manually perturb the shaft and observe spring-like recovery to the setpoint.

Both tests reproduce the canonical behaviours of any working FOC implementation. This was created in replication of the same reference example in SimpleFOC to confirm MyFOC is working as planned and look out for any possible initial defects by checking: encoder direction, alignment offset, current-sense polarity, PID gains, etc.

5.2.3. Clinical Exercise-Mode Procedure

The rehabilitation application is driven by a dedicated demonstration sketch whose serial interface exposes the impedance controller as a set of single-letter commands, so an exercise mode can be selected and parameterised at run time without re-flashing. The mode is chosen with M0–M3 (impedance, velocity, angle or torque); K, B, L and A set the virtual stiffness, damping, torque ceiling and adaptive gain; I< τ > commands a constant torque and selects torque mode; V< ω > sets a velocity target and selects velocity mode; W<grams> records an applied weight into the log; and T<n> tags subsequent samples as belonging to test n. The controller boots in a passive state ($K = B = A = 0$, $v_q = 0$) so the arm is free until a mode is commanded.

The five clinical modes presented in Section 6.5 were each run from this interface. The isometric test (T1) selects impedance mode with a stiff spring and a raised ceiling (M0, $K = 1.0 \text{ N}\cdot\text{m}/\text{rad}$, $L = 0.30 \text{ N}\cdot\text{m}$), after which calibrated weights are hung and logged with W while the arm is held; the isotonic test (T2) issues stepped constant-torque commands (I 0.10 = $0.30 \text{ N}\cdot\text{m}$, which select torque mode with $K = 0$) while a fixed 200 g load hangs from the lever; the isokinetic test (T3) toggles the velocity set-point (V +2, V 0, V -2 rad/s in velocity mode); the adaptive test (T4) enables effort scaling on the gentle deployed baseline (M0, $K = 0.5 \text{ N}\cdot\text{m}/\text{rad}$, $L = 0.05 \text{ N}\cdot\text{m}$, $A = 2.0$); and the transparent back-drive test (T5) is run from the passive boot state ($v_q = 0$) with the small breakaway weights logged through W. Each run is tagged with the matching T command so the modes can be separated in the logged CSV.

5.3. Data Logging and Plotting Pipeline

A short Python script was written to read the serial port, identify the CSV header emitted by *motor.logCSVHeader()*, accumulates subsequent rows into a pandas DataFrame and produces the plots described in Section 6. The script is reproducible across machines and platforms and forms part of the project's measurement infrastructure.

6. Results and Validation

6.1. Compile-Time Verification

Before any hardware deployment the library is verified to compile cleanly with avr-g++ targeting the ATmega328P. The full compile command and result are:

```
avr-g++ -c -mmcu=atmega328p -O2 -Wall -Wextra \
-fno-exceptions -std=c++11 \
-I MyFOC/src MyFOC/src/*.cpp
```

All C++ source files compile without warnings. The same verification is repeated for the application sketch *Rehab_Device*, which compile cleanly. The full library footprint on AVR is approximately 21 KB of program memory and zero static SRAM beyond per-object instance state, leaving over 10 KB of flash headroom for application code.

6.2. Hardware Bring-Up and Auto-Detected Parameters

A bring-up diagnostic routine was used for first power-on of the motor. It executes a three-phase test: (i) baseline ADC scan with the gate driver disabled; (ii) ADC re-scan with the driver enabled at 50 % duty cycle, which produces zero phase-to-phase voltage and therefore zero current but wakes the chip from its standby state; (iii) a slow rotor sweep through eleven

electrical revolutions to count the encoder pulses per mechanical revolution. The empirically observed results were:

Table 5: Diagnostic outputs from the as-built bench.

Quantity	Measured value
Encoder pulses per mechanical revolution (auto-detected)	8379
Suggested encoder CPR setting	8192
Pole-pair check estimate after CPR correction	11.003 (target 11)
Current-sense offset, channel A (driver enabled, no current)	344.1 ADC counts
Current-sense offset, channel B (driver enabled, no current)	345.4 ADC counts
Current-sense offset (driver disabled)	1.1 / 1.0 ADC counts
nFAULT / nOCTW status during normal operation	HIGH / HIGH

Two observations from this table guided subsequent design adjustments. First, the auto-detected encoder count of 8379 per revolution matches the AMT103 set to 2048 pulses per revolution (which yields 8192 counts after $4\times$ quadrature decoding); the firmware was duly configured for $\text{CPR} = 8192$. Second, the current-sense amplifier outputs read at the floor of the ADC range (≈ 1 count) when the gate driver was disabled but woke to 344 counts ($= 3.3 \frac{\text{V}}{2} \approx 1.68 \text{ V} \rightarrow 0.336 \times \text{ADC full scale}$) the moment the driver was enabled. This confirmed that the DRV8302 breakout used in this work ties the chip's internal SLEEP signal to EN_GATE, so the current-sense amplifiers are only operational while the driver is enabled. The current-sense calibration was therefore restructured to run with the driver enabled at 50 % duty (producing zero net phase voltage and therefore zero current) so that the captured offsets correspond to the operating bias point.

6.3. Identified Motor Parameters

After the diagnostic step, the parameter-identification procedure was run with the motor mechanically unloaded but free to rotate. The full identification sequence (resistance step, inductance step, IMC current-loop auto-tune, back-EMF spin) completed in approximately twelve seconds and produced the values shown in Table 6.

Table 6: Empirical motor parameters from the identification procedure.

Quantity	Identified value
Sensor direction (after alignment)	CCW
Zero electrical angle (after $\pi/2$ correction)	5.83 rad (alignment-position dependent)
Pole-pair check, empirical estimate	11.05 (target 11, error 0.5 %)
Stator phase resistance R	11.16 Ω
d-axis inductance L_d (step response)	7.46 mH
Time constant $\tau = \frac{L}{R}$	0.668 ms
Permanent-magnet flux linkage λ_{PM}	0.0346 Wb
Torque constant $K_t = \frac{3}{2} \cdot pp \cdot \lambda_{PM}$	0.570 N·m/A

The torque constant $K_t = 0.570$ N·m/A and the flux linkage $\lambda_{PM} = 0.0346$ Wb fall well within the expected range for an eleven-pole-pair gimbal motor and provide a reliable basis for the impedance control mode of the rehabilitation application. The resistance $R = 11.16$ Ω sits within the typical gimbal-motor range and is internally consistent with the steady-state Kirchhoff balance during the back-EMF test, lending it high confidence. The values in Table 6 are those of the second of two identification runs: an initial run, taken before a direction-sign error in the velocity loop was corrected, spun the rotor close to its terminal speed during the back-EMF spin and substantially under-estimated the flux linkage; after the fix the procedure was re-run, and the corrected values reported here were adopted for the firmware and used for every result in this chapter.

The d-axis inductance measurement deserves more cautious treatment. At the $U_{test} = 2$ V step voltage used by the procedure, the steady-state current is $\frac{U}{R} \approx 0.18$ A; with the 5 m Ω shunt and 10 $\times$ gain that corresponds to an ADC excursion of only about 1.8 counts above the calibrated zero. The 63.2 % threshold of the step response is therefore approximately one ADC count above offset, which is comparable to the converter's quantization and noise. The reported time constant of about 0.67 ms (yielding $L_d = 7.46$ mH) is therefore most plausibly explained by stochastic threshold-crossing rather than by the actual electrical time constant of the winding - small gimbal motors of this class typically have L in the 100–500 μ H range, so the identified value remains more than an order of magnitude too high. Confirmation of this hypothesis is offered by the observation that the IMC-tuned current loop, when activated with this measured L, drove the system into a limit cycle even when the impedance controller

commanded zero torque - exactly the failure mode predicted when $Kp = L \cdot \omega_{bw}$ is computed on a grossly inflated L . For this reason the deployed firmware does not use the auto-tuned IMC gains at all: the d - and q -axis current controllers are given conservative fixed gains ($Kp = 0.05$ V/A, $Ki = 50$ V/A·s), and the rehabilitation results in this chapter were recorded in voltage mode with current sensing disabled, so these inner current loops are configured but inactive.

6.4. Functional Verification of the FOC Pipeline

Each algorithmic stage was verified independently during bring-up:

- Mathematical kernel. Clarke, Park, inverse Park, inverse Clarke and SVPWM were unit-tested in a host environment against analytic expressions. The lookup-based sin/cos approximation produces a maximum error below 1×10^{-4} relative to the host math library across $[0, 2\pi)$.
- PWM output. An oscilloscope on each of the three INH inputs with the driver enabled at fixed 50 % duty confirmed a carrier frequency of $31.4 \text{ kHz} \pm 1 \%$ and centre-aligned (phase-correct) waveforms across the two timers.
- Encoder decoding. Manual rotation through one full mechanical revolution incremented the pulse counter by 8192 counts ($= 4 \times 2048$), confirming the configured CPR. The auto-detect routine in the diagnostic procedure yielded 8379 counts in a slow motor-driven sweep - the small shortfall from 8192 (about 2 %) is attributable to the rotor not always tracking the stator field with zero slip during the slow open-loop sweep.
- Sensor alignment. After *initFOC()*, the serial monitor reported *sensor_direction* = *CCW* and an empirical pole-pair estimate of 11.05 (target 11), confirming both the directional mapping and the $\frac{\pi}{2}$ offset correction applied during the alignment procedure. The reported zero-elec-angle differed between successive boots according to the arbitrary mechanical position at which the alignment ramp terminated; this is the expected behaviour.

6.5. Application: Robotic Rehabilitation Devices

After making sure all of the preliminary procedures were successfully run, the setup is ready to be used to explore the rehabilitation application. A scoping note applies to all of the application results that follow: MyFOC's closed-loop current path - the d - and q -axis current PI loops, the IMC tuning of Section 2.5, and the cross-axis decoupling feed-forward - is implemented and was exercised on the bench, but it was not used to collect the application data. The rehabilitation recordings in this section were taken with the library running in its voltage-mode commutation path, in which the commanded q -axis voltage is applied at the measured rotor electrical angle through the inverse Park and space-vector PWM stages with no current feedback (the low-side current sensing was not linked for these runs). The figures and statistics that follow therefore characterise the impedance and force-rendering behaviour under voltage-mode commutation rather than under closed-loop current regulation; the identified inductance that drives the current-loop tuning, and the caution it warrants, are discussed in Section 6.3.

6.5.1. Clinical Exercise Modes: Five-Mode Validation

The rehabilitation application was validated by operating the device in five clinically recognised exercise modes, each obtained from the same impedance kernel by a different choice of controller parameters and selected at run time over the serial link without re-flashing. Rather than exposing raw stiffness and damping numbers, the controller presents the patient with a recognisable exercise; the five modes — isometric, isotonic, isokinetic, adaptive-resistance and transparent (back-drive) — were each exercised on the bench and are described below with a schematic of the physical configuration and the corresponding recorded behaviour. All five runs used the same loaded pendulum arm (lever radius $r = 0.12$ m from the shaft axis to the weight) and were recorded in the voltage-mode commutation path described above. Over the serial link the commands K , B , L and A set the impedance parameters, $I<\tau>$ commands a constant torque (and selects torque mode), $V<\omega>$ a velocity set-point (velocity mode), $M0-3$ select impedance / velocity / angle / torque mode, $W<grams>$ logs an applied weight into the CSV stream and $T<n>$ tags the active mode; the device boots passive with $K = B = A = 0$.

Isometric — sustained hold against load (Test 1). In the isometric mode the controller renders a stiff virtual spring about the start position (impedance mode, $K = 1.0 \text{ N}\cdot\text{m}/\text{rad}$, ceiling raised to $L = 0.30 \text{ N}\cdot\text{m}$) so the lever is held essentially stationary while calibrated weights of 50, 100, 150 and 200 g are hung from its end (Figure 8). Because the arm does not move, the holding q-axis voltage needed to oppose gravity is a direct, repeatable measure of the torque the device is producing, and the hung weight provides a parameter-free ground-truth for that torque.

Test 1 — Isometric (deadweight calibration)

Controller holds the lever stationary; calibrated weights are hung from it

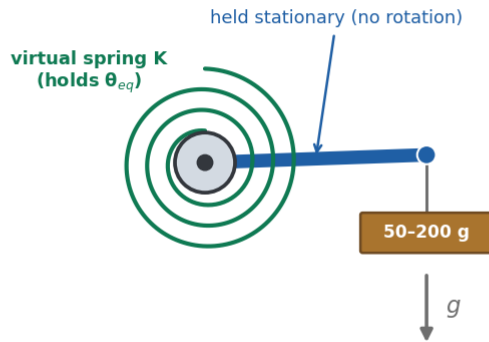


Figure 8: Setup for the isometric test (Test 1). The controller holds the lever level with a stiff virtual spring while calibrated weights are hung; the holding voltage and the known weight together give a torque measurement that is independent of the commanded control law.

Figure 9(a) shows the holding voltage rising in clean steps as each weight is added. Panel (b) is the key calibration: the true gravitational torque at the held angle, $m \cdot g \cdot r \cdot \cos(\varphi)$, is plotted against the magnitude of the commanded q-axis voltage. The three unsaturated points (50–150 g) fall on a straight line of slope $0.032 \text{ N}\cdot\text{m}/\text{V}$, whereas the voltage-to-torque

reconstruction used throughout this chapter, $\tau = v_q \cdot (\frac{K_t}{R})$ with $\frac{K_t}{R} = 0.0511 \text{ N}\cdot\text{m}/\text{V}$, is about $1.6\times$ steeper. Each of the three points lies below the reconstruction, the individual over-prediction ratios scattering between roughly $1.3\times$ and $2.1\times$. A through-origin regression of the three points places the deadweight slope at $0.032 \text{ N}\cdot\text{m}/\text{V}$ (95% confidence interval $0.029\text{--}0.036$) and the over-prediction factor at 1.6 (95% confidence interval $1.4\text{--}1.7$); the interval is wide because only three load levels were available, but the sign of the error — the open-loop voltage-mode reconstruction over-predicts the physically delivered torque at every point — is unambiguous. The 200 g point is excluded: at that load the lever drooped about 40° and the controller reached its torque ceiling, so it is saturated rather than a calibration point. This deadweight measurement is the only result in the chapter that is independent of the commanded law — the other four modes reconstruct torque from the same voltage they command, so they verify rendering fidelity but cannot detect an absolute-scale error of this kind. The discrepancy is precisely the consequence of operating open-loop in voltage mode, and closing the current loop (Section 7) is what would remove it.

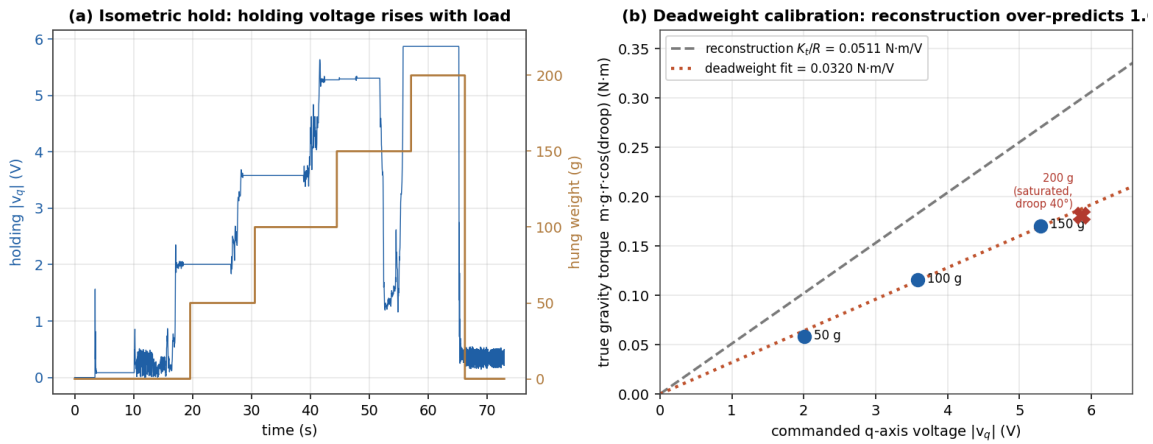


Figure 9: Isometric test (Test 1). (a) Holding q -axis voltage steps up as weight is added. (b) Deadweight calibration: the true gravitational torque (50–150 g) follows a $0.032 \text{ N}\cdot\text{m}/\text{V}$ line, about $1.6\times$ below the voltage-mode reconstruction $K_t/R = 0.0511 \text{ N}\cdot\text{m}/\text{V}$; the 200 g point is saturated (about 40° droop) and excluded.

Isotonic — constant resistive torque (Test 2). Sending $I\langle\tau\rangle$ places the controller in torque mode with the position spring disabled ($K = 0$) and commands a fixed q -axis torque; a fixed 200 g weight hangs from the lever, which is then free to rotate until the gravitational torque of that load balances the commanded torque (Figure 10). Stepping the command from roughly 0.10 to $0.30 \text{ N}\cdot\text{m}$ and letting the arm settle at each level demonstrates the defining property of isotonic exercise: a resistance that is constant with joint angle.

Test 2 — Isotonic (constant torque)

Constant torque lifts a fixed 200 g load to an equilibrium angle

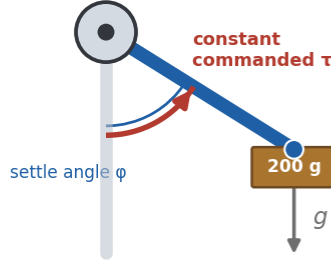


Figure 10: Setup for the isotonic test (Test 2). With the position spring disabled a constant torque is commanded and the lever rotates until the gravitational torque of the fixed 200 g load balances it; the settle angle is recorded against the commanded torque.

Figure 11(a) shows the reconstructed torque held at a sequence of flat levels as the command is stepped, and panel (b) plots the angle at which the arm settled against the commanded torque. The settle angle rises monotonically from about 28° to 62° as the resistance is increased, confirming that the larger commanded torque lifts the fixed 200 g load to a steeper equilibrium angle, where the motor torque balances the load's gravitational torque ($\tau = m \cdot g \cdot r \cdot \sin \phi$). The absolute torque values inherit the $1.6\times$ voltage-mode scale factor identified in the isometric test; the demonstration here concerns the constant-torque behaviour rather than absolute calibration.

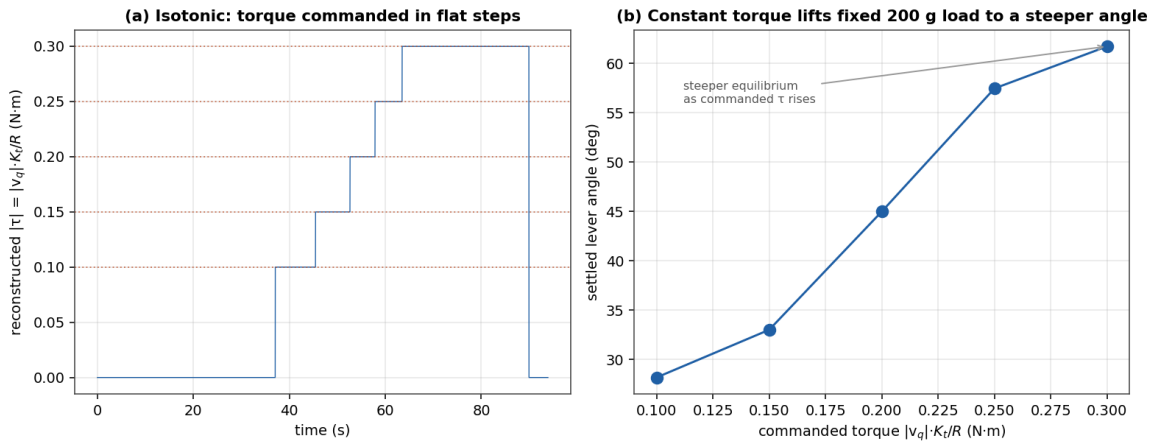


Figure 11: Isotonic test (Test 2). (a) Reconstructed torque commanded in flat steps. (b) The constant torque lifts a fixed 200 g load to a progressively larger settle angle (about 28° to 62°) as the command is increased from 0.10 to 0.30 N·m.

Isokinetic — constant angular velocity (Test 3). Selecting velocity mode with a set-point of +2, 0 and -2 rad/s drives the arm at a controlled speed regardless of the resisting load, the kinematic dual of the isotonic mode; the arm rotates continuously through several turns at each set-point (Figure 12).

Test 3 — Isokinetic (constant velocity)

Velocity set-point ± 2 rad/s drives the lever through continuous rotation

$$\omega = +2 \text{ rad/s (then 0, -2)}$$

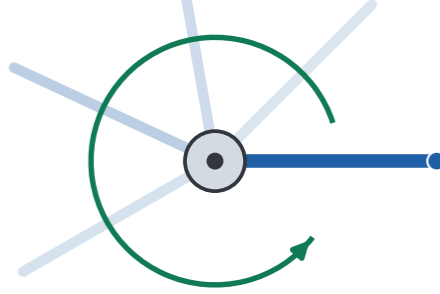


Figure 12: Setup for the isokinetic test (Test 3). A velocity set-point of ± 2 rad/s drives the arm at constant speed through continuous rotation.

Figure 13 shows the measured shaft velocity (raw and 0.4 s moving average) against the ± 2 rad/s set-point. Averaged over each interval the speed tracks the command well (mean $+2.20$ and -2.02 rad/s), but the instantaneous estimate is visibly noisy: the 1 kHz finite difference of the 8192-count encoder is coarse at these low speeds, and in the open-loop voltage path the velocity controller produces the bang-bang-like excursions seen in the trace rather than a smoothly regulated speed. The mode is therefore functional at the level of average velocity, with regulation quality that a closed current loop and a higher loop rate (Section 7) would improve.

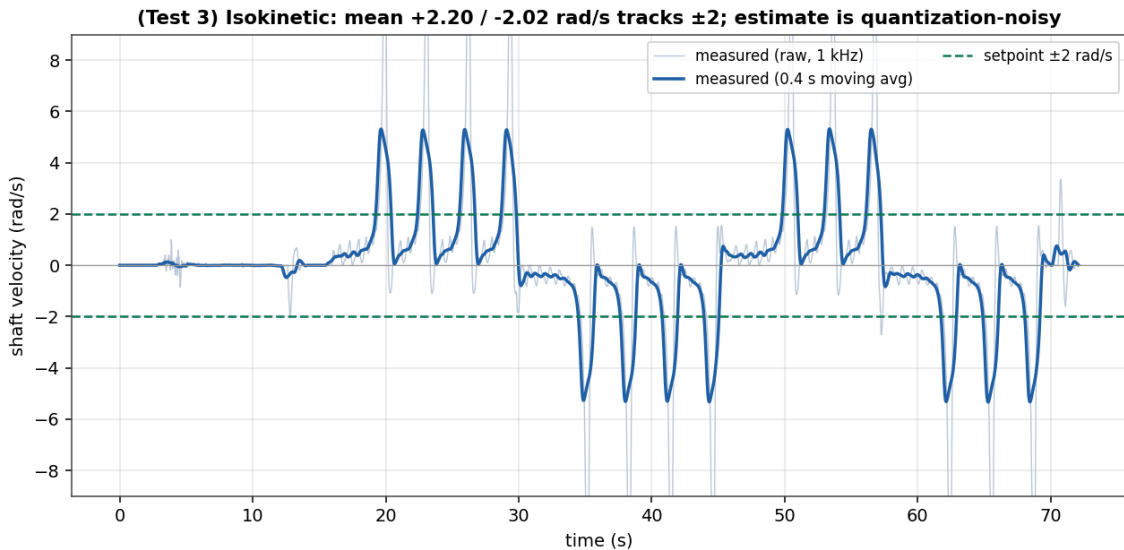


Figure 13: Isokinetic test (Test 3). Measured shaft velocity (raw and 0.4 s moving average) against the ± 2 rad/s set-point; the mean tracks the command ($+2.20 / -2.02$ rad/s) while the instantaneous estimate is quantisation-noisy.

Adaptive resistance (Test 4). The adaptive mode exercises the velocity-gated scaling at the deployed operating point: in impedance mode the stiffness and ceiling grow with the

operator's effort once the adaptive gain is enabled ($A = 2.0$), about the gentle deployed baseline $L_{base} = 0.05$ N·m (Figure 14). The run is deliberately split into two intervals — a baseline interval with adaptation disabled ($A = 0$) followed by an adaptive interval with $A = 2.0$ — so that the gating and the scaling law can both be checked on the same arm.

Test 4 — Adaptive (velocity-gated stiffness)

Stiffness scales with effort: $A = 0$ holds baseline, $A = 2$ stiffens with fast motion

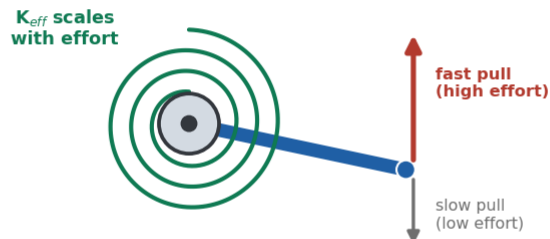


Figure 14: Setup for the adaptive test (Test 4). The operator pulls the arm slowly and then quickly; the effective stiffness and ceiling scale with the measured effort (low-pass-filtered speed while pushing outward).

Figure 15(a) overlays the effective stiffness and the effort metric against time across the two intervals. In the baseline interval ($A = 0$) the effort metric responds vigorously to hard pulls — reaching values near 9 — yet K_{eff} is pinned at its 0.5 N·m/rad baseline, confirming that the adaptation is correctly gated off. In the adaptive interval ($A = 2.0$) the same effort excursions drive K_{eff} up in bursts above 5 N·m/rad. Panel (b) plots K_{eff} against the effort metric: the baseline points sit on the 0.5 N·m/rad line, while the adaptive points fall on the analytic law $K_{eff} = K_{base} \cdot (1 + A \cdot \frac{effort}{v_{ref}})$ with $v_{ref} = 2.0$ rad/s, matching it to an RMS deviation of 3.3×10^{-4} N·m/rad over the 1362 adaptive-interval samples ($R^2 = 1.000$), with the fitted slope and intercept both equal to the design value of 0.5 to within ± 0.001 at 95% confidence. A patient who can move quickly is therefore met with progressively firmer resistance, while gentle motion is exercised against the soft baseline.

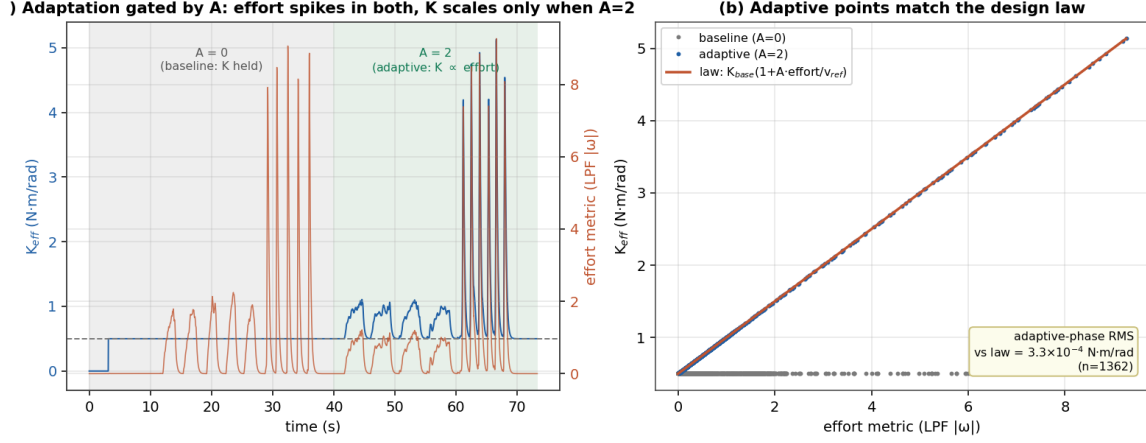


Figure 15: Adaptive test (Test 4) at the deployed baseline ($K_{base} = 0.5$ N·m/rad, $L_{base} = 0.05$ N·m, $A = 2.0$). (a) Effort and K_{eff} against time: with adaptation off ($A = 0$) effort spikes but K_{eff} stays at baseline; with $A = 2.0$ it scales with effort. (b) K_{eff} versus effort — the adaptive points match the law $K_{base}(1 + A \cdot \text{effort}/v_{ref})$ to an RMS of 3.3×10^{-4} N·m/rad ($n = 1362$).

Transparent back-drive (Test 5). In the passive boot state, the controller commands zero q-axis voltage ($v_q = 0$), so the motor produces no holding torque, and the arm is free to be moved by the patient — the transparent mode used for passive range-of-motion exercise (Figure 16). The de-energised arm was lifted and released to ring down freely, and small weights (15, 20 and 32 g) were then hung to find the static breakaway torque.

Test 5 — Backdrive (transparency)

Motor de-energised ($v_q = 0$); the lever swings freely and is probed for breakaway

motor OFF — $v_q = 0$
(no holding torque)

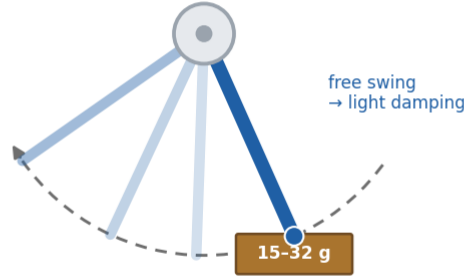


Figure 16: Setup for the back-drive test (Test 5). The motor is de-energised ($v_q = 0$); the free arm is released to ring down and small weights are added to find the breakaway torque.

Figure 17(a) shows the free swing of the de-energised arm: it oscillates for many seconds with only gradual amplitude loss, a log-decrement estimate over the free-swing placing the effective damping ratio in the lightly-damped range $\zeta \approx 0.02$ – 0.05 . Panel (b) gives the static breakaway torque inferred from the smallest weights that initiated motion, m·g·r, spanning 0.018–0.038 N·m. Together these confirm that the actuator is highly back-drivable, with a low residual friction floor — the property a transparent exercise mode requires.

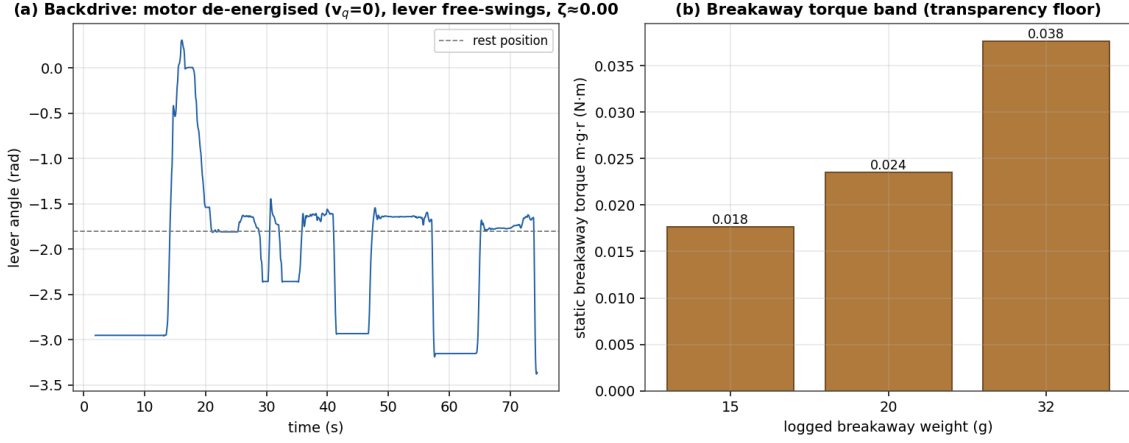


Figure 17: Back-drive / transparency test (Test 5). (a) The de-energised arm swings freely with light damping ($\zeta \approx 0.02$ – 0.05). (b) Static breakaway torque for the 15 / 20 / 32 g weights, 0.018–0.038 N·m.

Taken together, the five modes show that a single voltage-mode impedance controller, reparameterised at run time, reproduces the full set of clinically recognised exercise behaviours — sustained hold, constant resistance, constant velocity, effort-adaptive resistance and transparent free motion — on the same low-cost hardware. The one measurement independent of the commanded law, the isometric deadweight calibration, also quantifies the principal limitation of operating open-loop: the rendered torque tracks the programmed law in shape and reproducibility but carries an absolute-scale error of order $1.6\times$, which only the closed current loop of Section 7 can remove.

7. Limitations and Future Work

The preceding chapters establish that MyFOC implements a complete field-oriented control stack within the resource budget of an ATmega328P and that a single impedance kernel is sufficient to build a programmable rehabilitation application. Several limitations qualify how far those results extend, and they in turn define the most useful directions for future work. The most significant concerns the current-sensing path, which is implemented but is not yet exercised end-to-end in the application firmware.

7.1. Limitations

The library provides every element of a closed-loop current controller – a low-side current-sense interface, the forward Clarke and Park transforms applied to the measured phase currents, independent d- and q-axis current PI loops, Internal-Model-Control tuning, and a cross-axis decoupling feed-forward – but this closed-loop path was not used to collect any of the application results. The rehabilitation recordings were taken in the library’s voltage-mode commutation path, in which the commanded q-axis voltage is applied at the measured rotor electrical angle through the inverse Park and space-vector PWM stages with no current feedback. In this sense the current-sensing capability is present in the code base but is not yet fully supported as a validated operating mode.

Operating in voltage mode has two practical consequences. First, torque is produced open-loop through the relationship $v_q = R \cdot \left(\frac{\tau_{des}}{K_t} \right)$, which neglects the inductive $\frac{di}{dt}$ term and the speed-dependent back-EMF $\omega_e \cdot \lambda_{PM}$; the approximation is accurate only at low speed and

under quasi-static loading, so at higher speeds or during fast transients the achieved q-axis current, and therefore the rendered torque, departs from the commanded value. Second, without current feedback the firmware cannot enforce a true current limit; protection relies instead on the voltage ceiling and on the conservative torque bound derived from the phase resistance. The closed-loop current loop was brought up on the bench, but its tuning was hampered by the inductance-identification limitation described below, which produced a sustained limit cycle when the IMC gains were computed directly from the measured inductance.

The d-axis inductance is the least trustworthy of the four identified motor parameters, as already noted in Section 6.3. The electrical time constant $\frac{L}{R}$ of the small gimbal motor is short relative to both the current-sense resolution and the inner-loop period, so the step-based identification is dominated by stochastic threshold-crossing rather than by the true time constant. Because the IMC current-loop gains depend directly on the inductance, this uncertainty propagates straight into the current-loop tuning and is the proximate reason the closed-loop mode is not yet relied upon.

The target platform also bounds what the controller can achieve. The ATmega328P is an 8-bit device running at 16 MHz with no hardware floating-point unit, two kilobytes of SRAM and a ten-bit analog-to-digital converter; the field-oriented inner loop executes at roughly one to one-and-a-half kilohertz. This loop rate caps the achievable current-loop bandwidth and the temporal fidelity of fast transient effects, the emulated floating-point arithmetic consumes a substantial share of each loop, and the ten-bit converter limits current-sense resolution, compounding the inductance-identification difficulty above.

Two further limitations concern measurement and validation scope. The AMT103 encoder is incremental, so the controller requires an alignment and zeroing procedure at every power-up and retains a small residual offset from the open-loop alignment; it also provides no absolute position. Because the results were taken in voltage mode, the torque reported in Section 6.5 is reconstructed from the commanded q-axis voltage rather than measured with an independent transducer, so the close agreement with the programmed laws reflects rendering fidelity and internal consistency rather than an absolute torque calibration. The one independent check that was performed — the deadweight calibration of the isometric mode — bears this out quantitatively: the reconstructed torque over-predicts the true gravitational torque by a factor of roughly 1.6, so the absolute scale, as distinct from the shape and reproducibility of the rendered laws, is trustworthy only to within that factor until the current loop is closed. The most probable origin of this scale error is the torque constant itself. Because $K_t = 1.5 \cdot p \cdot \lambda_{PM}$ is derived from the permanent-magnet flux linkage λ_{PM} , and λ_{PM} is in turn identified from a back-EMF measurement taken while the motor is spinning, any resistive or friction-related voltage that is not fully separated from the open-circuit back-EMF inflates λ_{PM} and propagates directly into K_t and hence into the reconstruction $\tau = v_q \cdot \frac{K_t}{R}$. The observed factor corresponds to λ_{PM} (identified at 0.0346 Wb) being high by roughly 1.5–1.6 — a true value nearer 0.022 Wb, or equivalently a true K_t nearer 0.37 N·m/A rather than the computed 0.57 N·m/A. Accounting for the discrepancy through the resistance instead would require a true phase resistance near 17 Ω against the identified 11 Ω — some six ohms of extra series resistance that the inverter on-resistance and wiring cannot plausibly supply — so an over-estimated torque constant is the more likely dominant cause, with any resistance under-

estimation a smaller secondary contributor. The lever radius enters the gravitational torque linearly and is measured directly, so a geometric error large enough to explain the factor is unlikely. The hypothesis is straightforward to confirm on the bench by measuring K_t directly from a locked-rotor torque-versus-current characteristic and the phase resistance with a four-wire ohmmeter; either measurement would isolate the cause, and closing the current loop would in any case replace the open-loop voltage-to-torque mapping with a calibrated current-controlled torque. Finally, all results were obtained from a single gimbal motor on the bench, without mechanical load, thermal or long-duration testing; nor was a repeatability study performed, since every result derives from a single bench session, so run-to-run variance and the uncertainty of the identified motor parameters are not quantified; and the rehabilitation application has not undergone clinical or human-subject evaluation.

7.2. Future Work

The clearest priority is to bring the closed-loop current path into routine use. This entails linking the low-side current sense in the application firmware, re-tuning the d- and q-axis current loops against a trustworthy inductance estimate, and verifying closed-loop torque regulation against a commanded current reference. Doing so would replace the open-loop voltage-to-torque mapping with true current-controlled torque – improving accuracy at speed and during transients and enabling feedback-based current limiting – and would make the impedance rendering independent of the actuator’s electrical dynamics. Resolving the limit cycle observed during bring-up, whether through anti-windup refinement, a more conservative bandwidth, or a better inductance estimate, is the gating step.

Improving the parameter identification is closely tied to the previous point. A higher-resolution current-sense path – obtained through a higher gain setting on the DRV8302’s internal amplifier or through an oversampling-and-decimation scheme on the ATmega328P’s converter – would let the short $\frac{L}{R}$ time constant of small gimbal motors fall within the converter’s resolution. Alternatively, a signal-injection method that measures the current response to a high-frequency excitation would estimate the inductance without relying on a fast step response. Either route would directly enable the current-loop tuning called for above.

Porting the library to a faster 32-bit microcontroller with a hardware floating-point unit, higher converter resolution and a faster control loop – of the kind already supported by portable FOC libraries – would raise the achievable current-loop bandwidth and the fidelity of fast transient effects, while leaving the library’s structure and interfaces unchanged. The present code deliberately isolates the platform-specific timer, converter and pin handling, so such a port is expected to be confined to a small number of modules.

Several functional extensions follow naturally once closed-loop current control is in place. Field-weakening, implemented through a non-zero d-axis current reference, would extend the usable speed range; an absolute or sensorless position scheme would remove the boot-time alignment; and additional rehabilitation modes, together with mechanical-load and thermal characterisation, would broaden the validation envelope. For the rehabilitation application specifically, characterisation against a human operator and validation of the adaptive-scaling thresholds against patient-specific recovery data remain the principal outstanding work.

7.3. Path to Clinical Deployment

The bench system described in this thesis is a research and teaching platform; turning it into a device that can be used on patients in a clinic is a substantial change of envelope rather than a change of architecture. The mathematics, alignment procedure, impedance kernel and adaptive-scaling overlay all transfer unchanged, and only three classes of hardware need to scale up — the actuator, the sensing chain and the host processor — together with a safety and regulatory layer that does not exist on the bench at all. The remainder of this subsection sketches what each of those entails and quantifies the resulting parts-cost increase.

The bench actuator, a GM4108-120T gimbal motor, produces approximately $0.42 \text{ N}\cdot\text{m}$ at its motor-limited current ceiling. Upper-limb rehabilitation at the shoulder or elbow typically calls for $5\text{--}15 \text{ N}\cdot\text{m}$ at the joint, an order of magnitude more, so the actuator must be replaced. Two practical paths exist. A quasi-direct-drive (QDD) actuator such as the T-Motor AK80-9 (approximately USD 400, around $9 \text{ N}\cdot\text{m}$ peak with integrated driver and encoder) keeps the system back-drivable and adds little mechanical complexity, at the cost of moderate torque density and limited continuous rating. A frameless brushless servo such as the Kollmorgen or RoboDrive ILM-50x08 class (USD 600–2000) paired with a low-backlash harmonic-drive gearbox (USD 1500–4000) offers higher torque density and a more conventional industrial pedigree, at the cost of reduced transparency and increased mechanical complexity. The DRV8302 gate driver used on the bench is sufficient for the small gimbal motor but undersized for clinical actuators; an ODrive S1 (USD 200) or moteus n1 (USD 130) is appropriate for the QDD path, while an industrial servo drive (USD 500–2000) suits the geared path.

The single most consequential addition for clinical use is an in-line torque sensor — a rotary strain-gauge transducer placed between the actuator output and the patient interface, with a typical cost of USD 1500–5000. This sensor would close the torque loop directly on a calibrated mechanical measurement and would eliminate the $1.6\times$ absolute-scale error documented in Section 6.5 at its root, without requiring resolution of the open-loop voltage-mode reconstruction discussed in Sections 7.1 and 6.5. The incremental AMT103 should be replaced by an absolute device — an AS5048A magnetic encoder (USD 20) or an AMT22 absolute incremental (USD 35) for cost-sensitive work, or a higher-resolution absolute optical encoder (USD 200–500) for clinical-grade repeatability — so that the device boots into a known joint configuration without the alignment ramp the bench requires. The ATmega328P should give way to a 32-bit Cortex-M with a hardware FPU and 12-bit ADC: a Teensy 4.1 (USD 30) or an STM32H7 development board (USD 50–200) raises the current-loop rate from the bench’s $1\text{--}1.5 \text{ kHz}$ to a clinically conventional $10\text{--}20 \text{ kHz}$, restoring the bandwidth needed for higher-fidelity impedance rendering. The library’s platform-specific code is confined to three modules (MyBLDCDriver3PWM, MyEncoder and MyCurrentSense3PWM), so the port is bounded.

The safety layer changes character. A clinical device must include a mechanically-latching hardware emergency stop accessible to both patient and therapist, an independent watchdog implemented on a separate microcontroller, a mechanical torque limiter such as a slip clutch on the output shaft, a medical-grade isolated power supply, and sanitisable patient-contact surfaces. Beyond hardware, the device falls under IEC 60601-1 (general medical electrical safety), IEC 60601-1-2 (electromagnetic compatibility) and ISO 14971 (risk management);

achieving certification typically dominates the total cost of bringing a clinical device to market, often in the range of USD 50,000 to USD 500,000 for a first device. Those costs are not parts costs and are not included in the comparison below.

Table 7 compares per-axis parts cost for three configurations: the bench presented here, a low-cost clinical-grade configuration suitable for early-stage prototype evaluation, and a mid-range configuration representative of what a serious clinical prototype would use.

Table 7: Indicative per-axis parts cost (USD) for the bench presented here and two clinical-grade configurations. Excludes certification, software licences, and labour.

Component	Bench (this work)	Clinical low	Clinical mid
Motor	\$40 (GM4108-120T gimbal)	\$400 (T-Motor AK80-9 QDD)	\$1200 (frameless servo)
Gearbox	none (direct drive)	none (QDD)	\$2500 (harmonic drive)
Driver	\$30 (DRV8302 breakout)	\$200 (ODrive S1)	\$800 (industrial servo drive)
MCU	\$15 (UNO clone)	\$30 (Teensy 4.1)	\$200 (STM32H7 board)
Position sensor	\$30 (AMT103 incremental)	\$20 (AS5048A absolute)	\$200 (absolute optical)
Torque sensor	none	\$1500 (strain-gauge)	\$4000 (rotary transducer)
Safety hardware	none	\$200 (E-stop, watchdog)	\$800 (full safety chain)
Mechanical / ergonomic	\$20 (lever, weights)	\$500 (handle, mount)	\$2000 (clinical frame)
Per-axis subtotal	~\$135	~\$2,875	~\$11,700

The per-axis numbers in Table 7 scale roughly linearly with the number of actuated joints. A representative three-degree-of-freedom upper-limb device (shoulder, elbow and wrist) would therefore land in the range of USD 9,000 to USD 35,000 in parts alone, depending on configuration. By comparison, commercial upper-limb rehabilitation systems — Hocoma ArmeoPower and Bionik InMotion Arm being the best-known examples — retail for roughly USD 150,000–200,000 and USD 80,000–150,000 respectively. The order-of-magnitude gap between parts cost and retail price reflects the certification, clinical validation, integration and warranty costs noted above, not the cost of the hardware itself.

What MyFOC contributes in this context is not a clinical device but the entry barrier to one. The bench at USD 135 is roughly twenty times cheaper than the cheapest clinical-grade configuration and roughly a thousand times cheaper than a commercial endpoint, so a student or research group can explore impedance control, adaptive scaling and the algorithmic side of rehabilitation robotics — including the mathematical and control issues that dominate the early prototype stage — before incurring any of the cost of the clinical pathway. The same library code, with its platform-specific layer replaced, then ports forward into the clinical prototype rather than being discarded.

8. Conclusion

This project has presented MyFOC, a Field-Oriented Control library implemented from first principles for the Arduino UNO + DRV8302 platform, together with a rehabilitation application built on its impedance kernel. The library was developed without reference to existing FOC firmware beyond the well-documented mathematical foundations, and the resulting code base of approximately 21 KB of flash and zero static SRAM beyond per-object instance state validates the claim that a full FOC stack - including parameter identification, IMC current-loop tuning, sensor alignment and an impedance controller - is implementable within the constraints of an ATmega328P-class device while leaving substantial headroom for application logic.

The application study presented in Section 6.5 demonstrates that the same impedance kernel suffices to render a programmable virtual spring–damper field with a hardware-enforced safety ceiling, and that a velocity-gated adaptive-scaling overlay can be added without disrupting the underlying loop. The controller was exercised as five clinically recognised exercise modes — isometric, isotonic, isokinetic, adaptive-resistance and transparent back-drive — on the bench hardware. In the adaptive mode the measured effective stiffness tracked the velocity-gated design law to an RMS of $3.3 \times 10^{-4} \text{ N}\cdot\text{m}/\text{rad}$ over 1362 samples while remaining correctly held at the baseline when the adaptation was gated off, confirming that the firmware implements the law without numerical degradation. An independent deadweight calibration performed in the isometric mode showed that the open-loop voltage-mode torque reconstruction over-predicts the physically delivered torque by a factor of roughly 1.6, the clearest single measure of the absolute-scale error that closing the current loop would remove. These results were obtained in the library’s voltage-mode commutation path, with the current/torque loop open; bringing the implemented closed-loop current controller into validated use is the foremost item of future work (Section 7).

Of the limitations and extensions set out in the preceding chapter, two are the most immediately actionable. First, the inductance-identification routine identified in Section 6.3 as the least-trusted of the four motor parameters would benefit from a higher-resolution current-sense path, either through a higher gain setting on the DRV8302’s internal amplifier or through an oversampling scheme on the ATmega328P’s ADC, so that the L/R time constant of small gimbal motors falls comfortably within the converter’s resolution. Second, the rehabilitation application as presented operates without clinical evaluation; the existing torque-ceiling and adaptive-scaling primitives are designed to be safe and clinically expressive, but characterisation against a human operator and validation of the adaptive thresholds against patient-specific recovery curves remain future work.

The contribution of this work is twofold. First, it demonstrates that a complete field-oriented control stack — motor parameter identification, Internal-Model-Control current-loop tuning, encoder alignment, space-vector commutation and an impedance controller — can be implemented from first principles within the resource budget of an 8-bit ATmega328P, in roughly 21 KB of flash and with no static SRAM beyond per-object state, while leaving headroom for application logic and remaining portable to faster microcontrollers without structural change. Second, it shows that a single impedance kernel, exposed through a compact serial command set, suffices to render five clinically recognised rehabilitation exercise modes — isometric, isotonic, isokinetic, adaptive-resistance and transparent back-drive — on that hardware, including a velocity-gated adaptive-stiffness overlay that tracks its design law to an RMS of 3.3×10^{-4} N·m/rad. A methodological contribution accompanies these: the library is built and documented transparently for reproduction, and its voltage-mode operating limits are characterised honestly, including a parameter-free deadweight calibration that quantifies the open-loop torque-reconstruction error.

The artifact and its accompanying documentation are intended as a pedagogically transparent and reproducible reference platform on which any of the above directions can be pursued.

References

- [1] Microchip Technology Inc., ATmega328P 8-bit AVR Microcontroller Datasheet, document DS40002061B, August 2020.
- [2] Texas Instruments, DRV8302 / DRV8303 Three-Phase Pre-Driver with Dual Current Shunt Amplifiers and Buck Regulator, datasheet SLOS738B, Texas Instruments Inc., revised May 2016.
- [3] E. Clarke, Circuit Analysis of A-C Power Systems, vol. 1, John Wiley & Sons, New York, 1943.
- [4] R. H. Park, "Two-reaction theory of synchronous machines - Generalized method of analysis," Transactions of the AIEE, vol. 48, no. 3, pp. 716–727, July 1929.
- [5] D. G. Holmes and T. A. Lipo, Pulse Width Modulation for Power Converters: Principles and Practice, IEEE Press / Wiley, Hoboken NJ, 2003.
- [6] J. Holtz, "Pulsewidth modulation for electronic power conversion," Proceedings of the IEEE, vol. 82, no. 8, pp. 1194–1214, August 1994.
- [7] S. Skogestad and I. Postlethwaite, Multivariable Feedback Control: Analysis and Design, 2nd ed., John Wiley & Sons, Chichester, 2005.
- [8] N. Hogan, "Impedance control: An approach to manipulation: Part I - Theory; Part II - Implementation; Part III - Applications," ASME Journal of Dynamic Systems, Measurement, and Control, vol. 107, no. 1, pp. 1–24, March 1985.
- [9] N. Hogan and S. P. Buerger, "Impedance and interaction control," in Robotics and Automation Handbook, T. R. Kurfess, Ed., CRC Press, 2005, ch. 19.
- [10] CUI Devices, AMT10 Series Modular Incremental Encoder Datasheet, March 2021.
- [11] A. Skuric, H. S. Bank, R. Unger, O. Williams and D. González-Reyes, "SimpleFOC: A field oriented control (FOC) library for controlling brushless DC (BLDC) and stepper motors," Journal of Open Source Software, vol. 7, no. 74, p. 4232, 2022.
- [12] R. Krishnan, Permanent Magnet Synchronous and Brushless DC Motor Drives, CRC Press, Boca Raton FL, 2010.
- [13] W. Leonhard, Control of Electrical Drives, 3rd ed., Springer, Berlin, 2001.

Appendix A. Source-File Organisation

The MyFOC library directory structure is summarised below. *Italic entries are documentation*; **bold entries are required for the library to build under the Arduino IDE**. The tree mirrors the layout produced by the deliverable archive.

MyFOC/

— library.properties	[Arduino library manifest]
— README.md	[user-facing quick-start]
— DOCUMENTATION.md	[detailed math + design notes]
— src/	
— MyFOC.h	[umbrella include]
— MyFOCMath.h, .cpp	[transforms, SVPWM, sin/cos LUT]
— MyPID.h, .cpp	[discrete-time PID]
— MyLPF.h, .cpp	[first-order low-pass filter]
— MyEncoder.h, .cpp	[quadrature decoder]
— MyBLDCDriver3PWM.h, .cpp	[Timer1/2 PWM driver]
— MyCurrentSense3PWM.h, .cpp	[DRV8302 current sense]
— MyBLDCMotor.h, .cpp	[FOC engine and control modes]
— MyMotorID.h, .cpp	[R, Ld, lambda_PM identification + IMC tuning]
— examples/	
— Diagnostic/	[wiring check & encoder-CPR auto-detect]
— Motor_Identification/	[R, Ld, lambda_PM parameter identification]
— Rehab_Device/	[impedance-control rehab application]

To install, the entire MyFOC directory is copied into the Arduino IDE's libraries folder (typically ~/Arduino/libraries on Unix-like systems and Documents\Arduino\libraries on Windows). The IDE then offers each example sketch under File → Examples → MyFOC. The application sketch that produced the empirical data in Section 6 is Rehab_Device. It follows a simple template: hardware pin definitions, identified-motor-parameter constants pasted from the parameter-identification procedure, the *setup()* routine that initialises the encoder, driver, current sense, FOC pipeline and impedance kernel, and the *loop()* routine that runs *loopFOC()*, applies the impedance or torque law, calls *move()*, and emits a CSV row

when logging is active. The sketch also exposes a one-character serial command interface for runtime parameter adjustment; the full command list is printed in response to *H* or *?*.

Appendix B. Mathematical Derivations

This appendix collects the derivations cited but not fully worked out in the main text. Each subsection is self-contained; readers familiar with the result may skip to the next subsection.

B.1 Clarke Transformation (Amplitude-Invariant Form)

Let the three stator phase axes be unit vectors in the $\alpha\beta$ plane:

$$a = (1, 0), \quad b = \left(-\frac{1}{2}, \frac{\sqrt{3}}{2}\right), \quad c = \left(-\frac{1}{2}, -\frac{\sqrt{3}}{2}\right)$$

The $\alpha\beta$ projection of any three-phase quantity (v_a, v_b, v_c) is the vector sum scaled by an arbitrary constant k :

$$v_{\alpha\beta} = k \cdot (v_a \cdot a + v_b \cdot b + v_c \cdot c)$$

Component-wise:

$$\begin{aligned} v_\alpha &= k \cdot \left(v_a - \frac{v_b}{2} - \frac{v_c}{2}\right) \\ v_\beta &= k \cdot \left(v_b \cdot \frac{\sqrt{3}}{2} - v_c \cdot \frac{\sqrt{3}}{2}\right) = k \cdot \left(\frac{\sqrt{3}}{2}\right) \cdot (v_b - v_c) \end{aligned}$$

For amplitude invariance we require that a balanced sinusoidal input $(v_a, v_b, v_c) = V \cdot (\cos\theta, \cos(\theta - \frac{2\pi}{3}), \cos(\theta + \frac{2\pi}{3}))$ produces $|v_{\alpha\beta}| = V$. Substituting:

$$\begin{aligned} v_\alpha &= k \cdot V \cdot \left[\cos\theta - \frac{1}{2}\cos\left(\theta - \frac{2\pi}{3}\right) - \frac{1}{2}\cos\left(\theta + \frac{2\pi}{3}\right)\right] \\ &= k \cdot V \cdot \left[\cos\theta - \frac{1}{2} \cdot 2\cos\theta \cdot \cos\left(\frac{2\pi}{3}\right)\right] \\ &= k \cdot V \cdot \left[\cos\theta + \frac{1}{2}\cos\theta\right] = k \cdot V \cdot \left(\frac{3}{2}\right) \cdot \cos\theta \end{aligned}$$

Setting $k = 2/3$ yields $v_\alpha = V\cos\theta$, the amplitude-invariant convention. The analogous derivation for v_β yields $v_\beta = V\sin\theta$.

B.2 Park Transformation

The Park transform is a rotation by $-\theta_e$ of the $\alpha\beta$ vector into a frame whose d-axis coincides with the rotor's permanent-magnet direction. The rotation matrix is:

$$R(-\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Therefore:

$$\begin{aligned} v_d &= v_\alpha \cdot \cos\theta + v_\beta \cdot \sin\theta \\ v_q &= -v_\alpha \cdot \sin\theta + v_\beta \cdot \cos\theta \end{aligned}$$

The inverse transformation is $R(+\theta)$, used in the firmware to convert the dq command voltages back to $\alpha\beta$ before SVPWM.

B.3 PMSM Torque Equation

In the dq frame the instantaneous power delivered to the rotor is:

$$P = \left(\frac{3}{2}\right) \cdot (v_d \cdot i_d + v_q \cdot i_q)$$

Substituting the dq voltage equations of Section 2.2.3 and isolating the mechanical power-converting terms (those linear in ω_e):

$$P_{mech} = \left(\frac{3}{2}\right) \cdot \omega_e \cdot [(L_d - L_q) \cdot i_d \cdot i_q + \lambda_{PM} \cdot i_q]$$

Since $P_{mech} = \tau \cdot \omega_m$ and $\omega_e = pp \cdot \omega_m$:

$$\tau = \left(\frac{3}{2}\right) \cdot pp \cdot [\lambda_{PM} \cdot i_q + (L_d - L_q) \cdot i_d \cdot i_q]$$

For a surface PMSM with $L_d \approx L_q$ the second (reluctance) term vanishes, leaving:

$$\tau = \left(\frac{3}{2}\right) \cdot pp \cdot \lambda_{PM} \cdot i_q = K_t \cdot i_q$$

This is the relationship the impedance controller inverts when converting a commanded torque τ_{des} into a current-loop reference $i_{qref} = \frac{\tau_{des}}{K_t}$.

B.4 IMC Tuning of the First-Order Current-Loop Plant

After cross-axis decoupling, each axis of the dq current loop sees a first-order plant:

$$G(s) = \frac{1}{L \cdot s + R}$$

The Internal Model Control rule specifies a desired closed-loop transfer function:

$$T_{target}(s) = \frac{1}{\tau_{cl} \cdot s + 1}, \tau_{cl} = 1/\omega_{bw}$$

Solving for the controller $C(s)$ such that the closed-loop transfer function $G \cdot C / (1 + G \cdot C) = T_{target}$ yields:

$$\begin{aligned} C(s) &= T_{target} / (G \cdot (1 - T_{target})) \\ &= (L \cdot s + R) \cdot \frac{\omega_b}{s} \\ &= L \cdot \omega_{bw} + R \cdot \frac{\omega_{bw}}{s} \end{aligned}$$

which is a PI controller with $K_p = L \cdot \omega_{bw}$ and $K_i = R \cdot \omega_{bw}$. These are the gains computed by `tuneCurrentLoops( $\omega_{bw}$ )` once R and L have been identified.

B.5 Impedance Dynamics for a Single-Degree-of-Freedom Load

Consider the actuator and any rigid load as a single degree of freedom with effective rotational inertia J_{eff} . Under the spring–damper torque law $\tau_{act} = -K \cdot (\theta - \theta_{eq}) - B \cdot \omega$, Newton’s second law for rotation gives:

$$J_{eff} \cdot \frac{d^2\theta}{dt^2} = \tau_{ext} + \tau_{act} = \tau_{ext} - K \cdot (\theta - \theta_{eq}) - B \cdot \left(\frac{d\theta}{dt}\right)$$

With $\tau_{ext} = 0$ (free response after a perturbation), the dynamics reduce to the canonical second-order linear ODE whose characteristic polynomial is:

$$J_{eff} \cdot s^2 + B \cdot s + K = 0$$

yielding the natural angular frequency $\omega_n = \sqrt{\frac{K}{J_{eff}}}$ and the damping ratio $\zeta = \frac{B}{2 \cdot \sqrt{K \cdot J_{eff}}}$. These quantities determine the shape of the push-response trajectories shown in Figures 10 and 11 of Section 6.5: under-damped responses ($\zeta < 1$) produce decaying oscillation, critically damped responses ($\zeta = 1$) settle in minimum time without overshoot, and over-damped responses ($\zeta > 1$) settle exponentially without oscillation.

In the velocity-adaptive variant of Section 6.5, the stiffness, damping and torque ceiling are scaled together by the common factor $\sigma = 1 + A \cdot \text{effort} / v_{ref}$. Because the natural frequency $\omega_n = \sqrt{(K/J)}$ and the damping ratio $\zeta = B/(2\sqrt{(KJ)})$ then both scale as $\sqrt{\sigma}$, a more vigorous patient meets a device that is not only stiffer but also faster and more heavily damped: the transient sharpens and its overshoot is reduced as effort rises, rather than retaining a fixed shape. The clinical interpretation is that a stronger patient encounters a firmer, more tightly regulated resistance, while the gentler and more lightly damped baseline dynamics are reserved for slower motion.